

IF69D-S11 e OPEET014-OPEL

Processamento Digital de Imagens

Aula 08 – Segmentação de Imagens

Universidade Tecnológica Federal do Paraná - UTFPR - Curitiba
Departamento Acadêmico de Eletrônica - DAELN
Curso de Engenharia Eletrônica
Prof. Gustavo B. Borba e Prof. Humberto R. Gamba

05.mai.2026

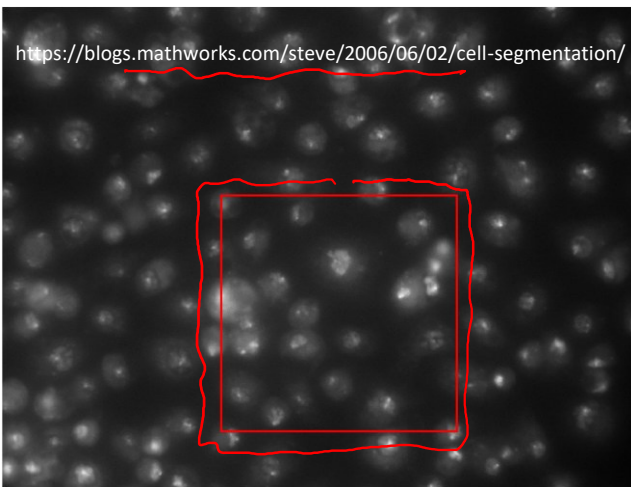
Image segmentation: O que segmentação ?

Objetivo da segmentação: separar automaticamente cada núcleo do fundo e dos demais.

Aplicações de segmentação: Separar automaticamente objetos em uma imagem, e avaliar o seu conteúdo, respondendo:

- Quantos objetos (células) existem na imagem? ✓
- Como separar objetos adjacentes? ✓
- Como definir automaticamente as fronteiras? ✓

Imagem de núcleos celulares obtida por microscopia de fluorescência.



Núcleos celulares (microscopia de fluorescência)

Recorte (ROI): foco em uma região para análise

Facilitar a visualização e o entendimento das técnicas de segmentação.

```
clc, clear, close all
I = imread('https://blogs.mathworks.com/images/steve/60/nuclei.png');
figure, imshow(I,[]);
hold on
rectangle('Position',[465 400 500 500], 'EdgeColor','r', 'LineWidth',2);
I_cropped = I(400:900, 465:965);
figure, imshow(I_cropped,[]);
```

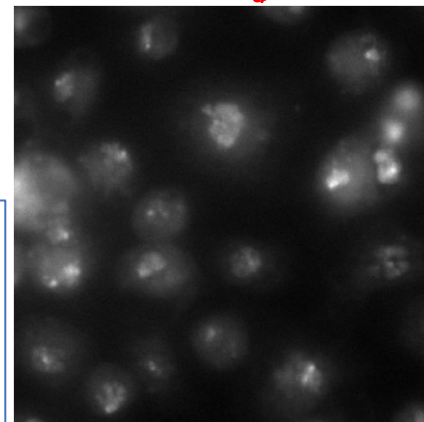
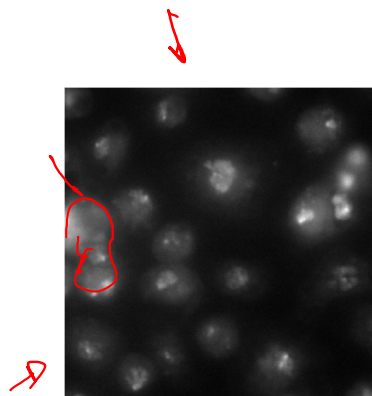
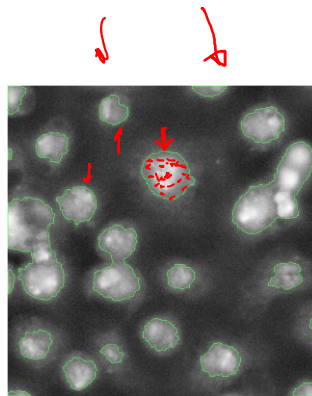


Image segmentation: O que é segmentação?

Objetivo: separar automaticamente objetos de interesse do fundo (e entre si).



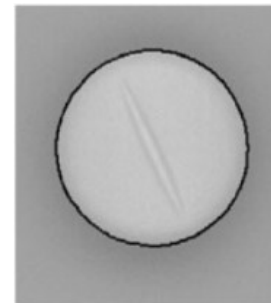
Núcleos celulares



Segmentação (contorno)



Inspeção industrial



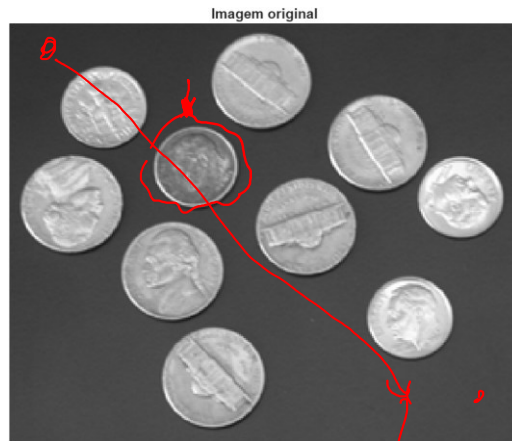
Contorno detectado

<https://blogs.mathworks.com/steve/2006/06/02/cell-segmentation/>

Miha Mozina, Dejan Tomazevic, Franjo Pernus, Bostjan Likar, Real-time image segmentation for visual inspection of pharmaceutical tablets, Machine Vision and Applications, v. 22, p. 145-156, 2011, DOI 10.1007/s00138-009-0218-7.

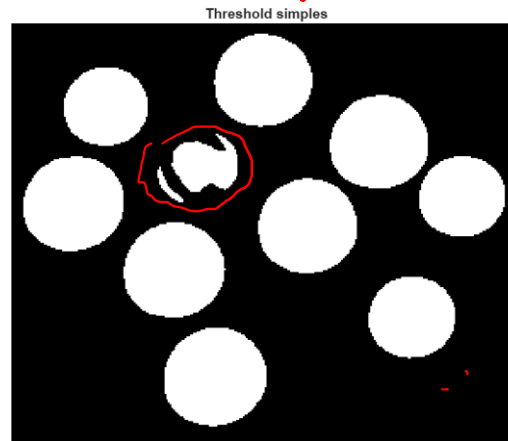
Image segmentation: O que é segmentação?

Exemplo de um pipeline de segmentação

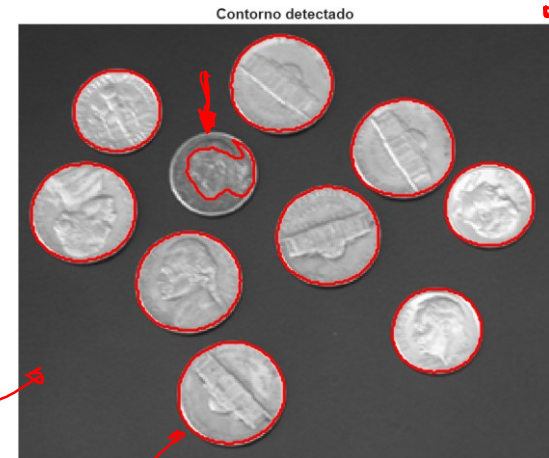
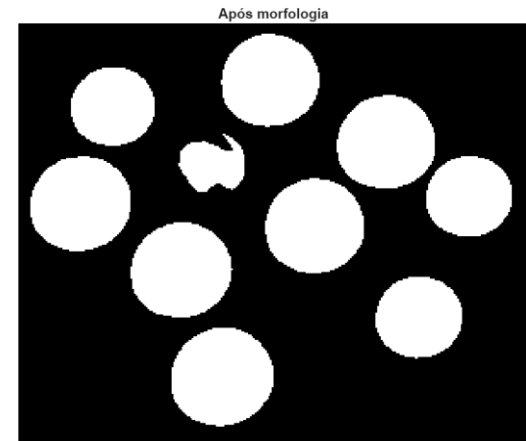


coins.png

variações
na iluminação



$T = 128$ BW



labelling

Image segmentation: Limiarização global (thresholding)

ideia: separar por intensidade, transformando uma imagem em binária:

$$g(x, y) = \begin{cases} 1 & f(x, y) > T \\ 0 & \text{caso contrário} \end{cases}$$

Handwritten notes: A red circle around the '1' in the first case, a red arrow pointing to the '0' in the second case, and a red 'N' above the first case.

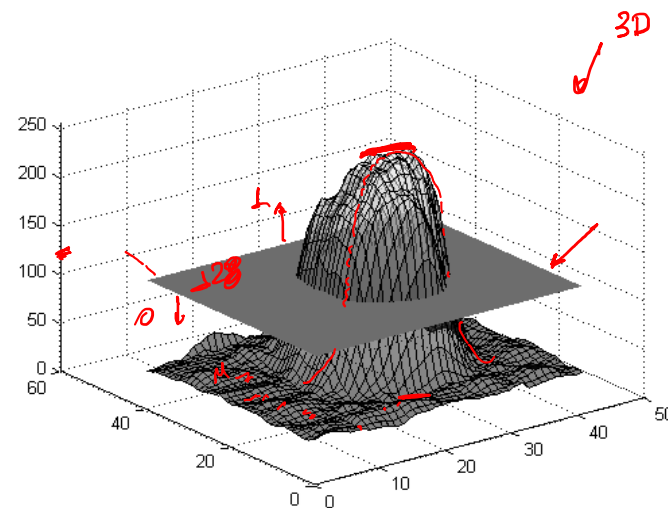
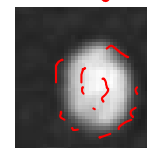
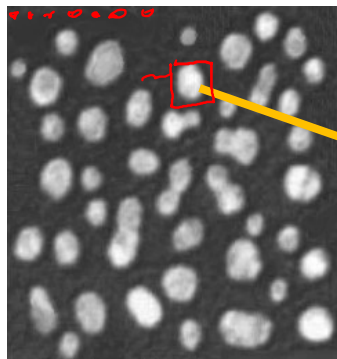


Image segmentation: Limiarização global (thresholding)

Qual separa melhor objeto e fundo?

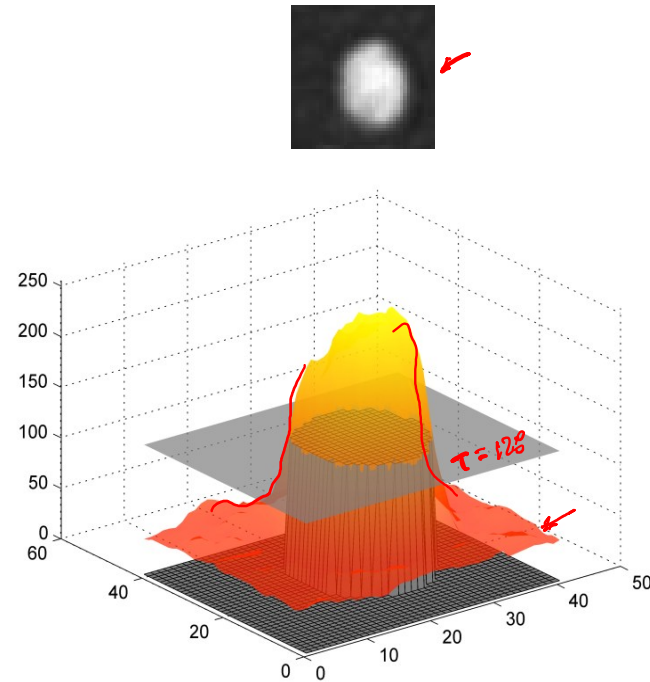
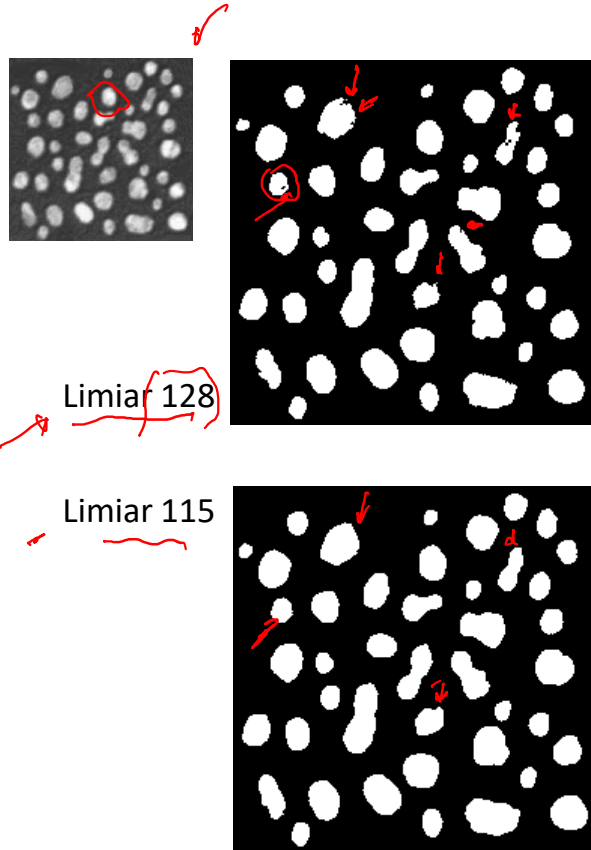
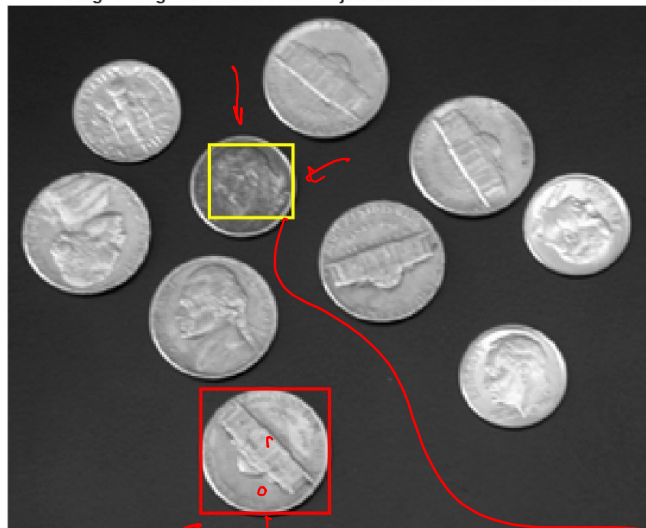
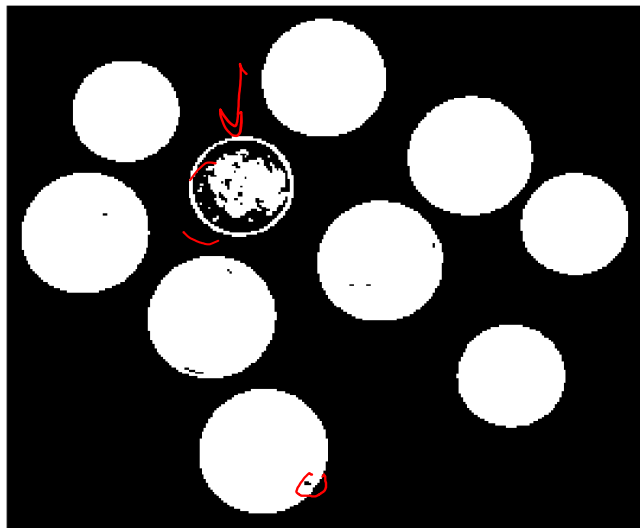


Imagem original: moeda boa e objeto com contorno não circular



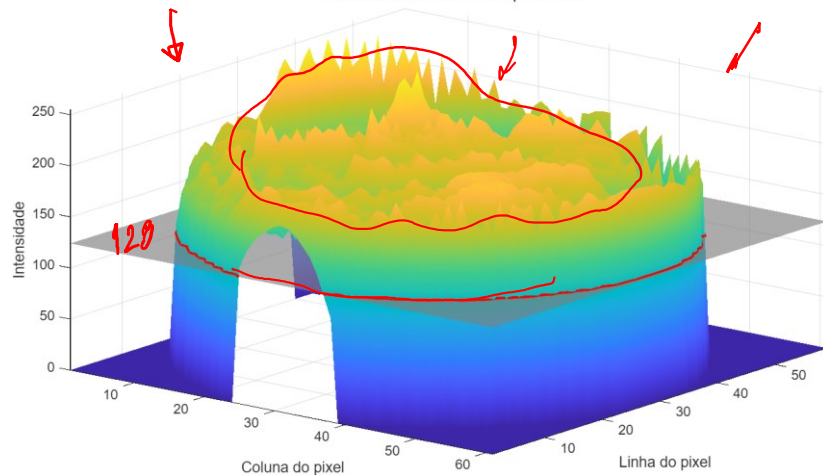
Threshold sem filtro



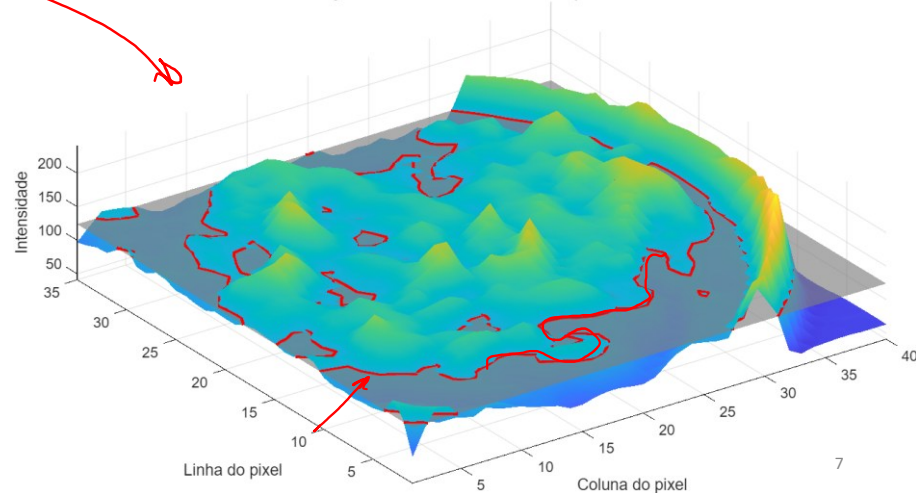
BW
↳ simão 1 objeto
o fundo

```
BW_no_filter = imbinarize(I, level);  
figure;  
imshow(BW_no_filter);  
title('Threshold sem filtro');
```

Moeda circular selecionada | T = 124.0



Objeto com contorno não circular | T = 124.0



Como encontrar o limiar ?

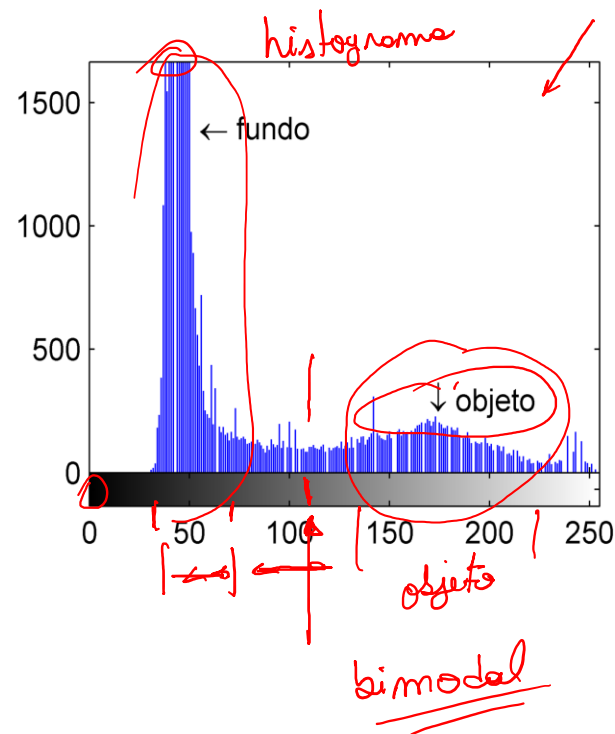
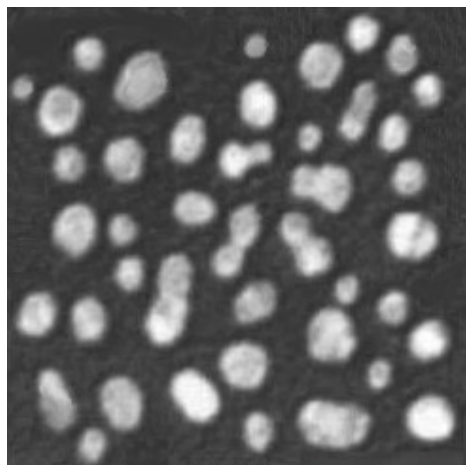
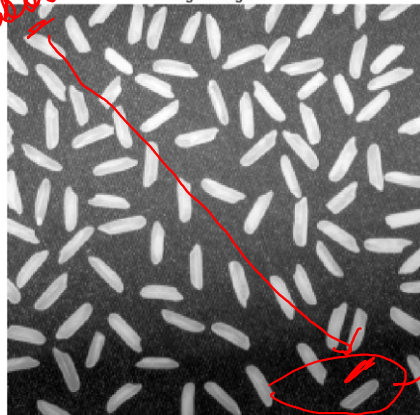


Image segmentation: Limiarização global (thresholding)

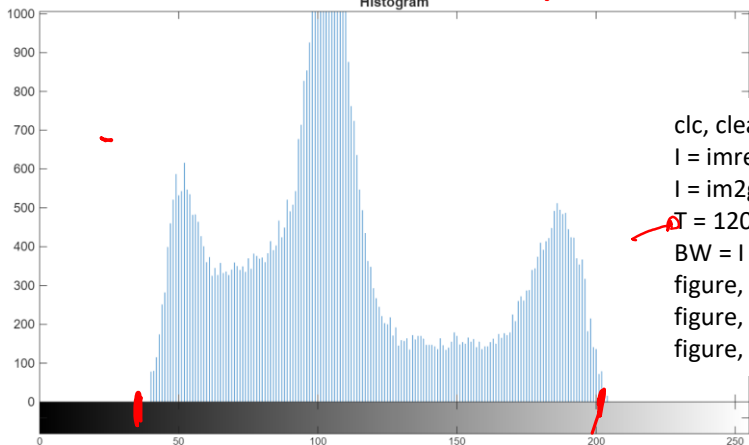
Quando a limiarização simples falha ?

- Iluminação não uniforme
- Baixo contraste
- Histograma não bimodal

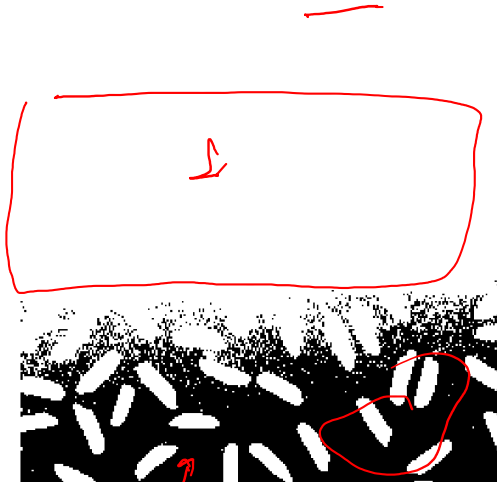
Imagem original



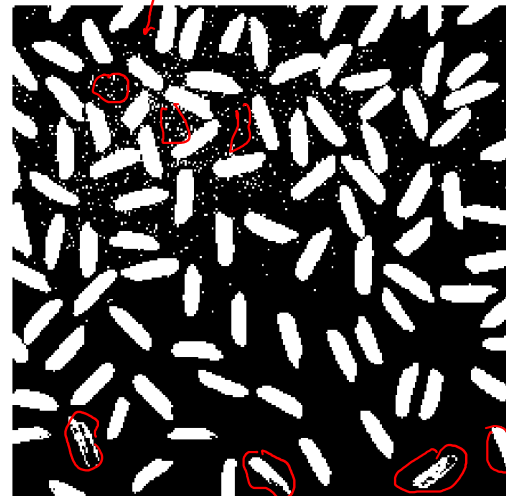
Histogram



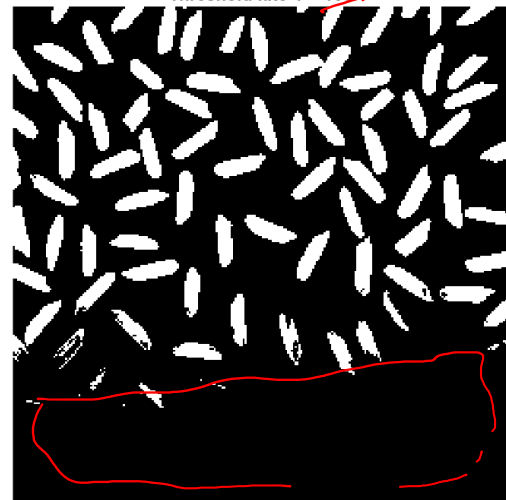
Threshold fixo $T = 80$



Threshold fixo $T = 120$



Threshold fixo $T = 160$

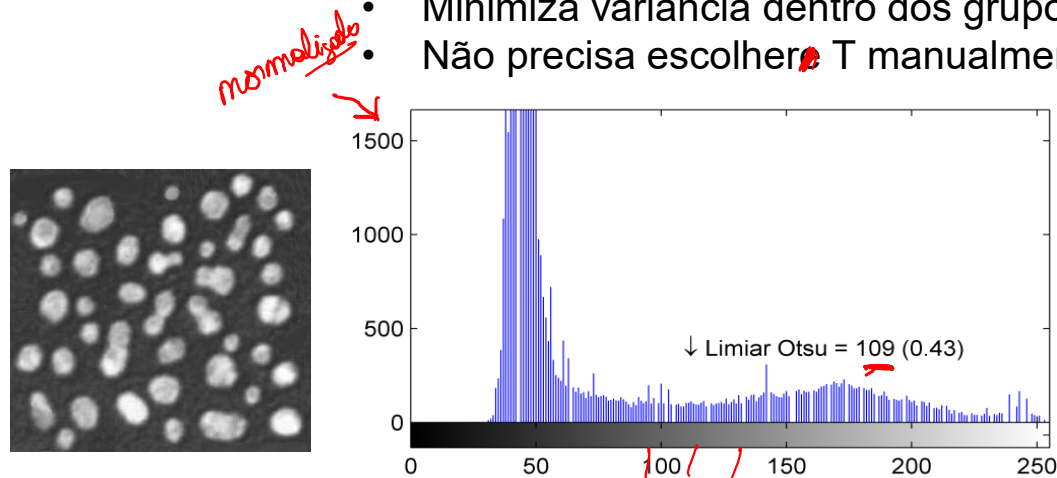


```
clc, clear, close all
I = imread('rice.png');
I = im2gray(I);
T = 120;
BW = I > T;
figure, imshow(I); title('Original');
figure, imshow(BW); title(['Threshold fixo T = ', num2str(T)]);
figure, imhist(I); title('Histograma');
```

Método de Otsu

Limiarização automática (Otsu)

- Divide pixels em dois grupos
- Minimiza variância dentro dos grupos
- Não precisa escolher T manualmente



$T = 109$

Otsu obtém o nível de cinza que melhor separa, segundo o critério estatístico da variância, os dois grupos de pixels.

O valor ótimo de limiar é aquele que minimiza a variância dentro de cada grupo pixels (pixels mais parecidos) σ_w^2 :

$$\sigma_w^2(t) = q_1(t)\sigma_1^2(t) + q_2(t)\sigma_2^2(t) \quad q_1 = \sum_{i=0}^{t-1} p(i) \quad q_2 = \sum_{i=t}^{L-1} p(i) \quad q_1 + q_2 = 1$$

Image segmentation: Limiarização global (thresholding)

Imagem original

coins.png



Histograma | T = 126

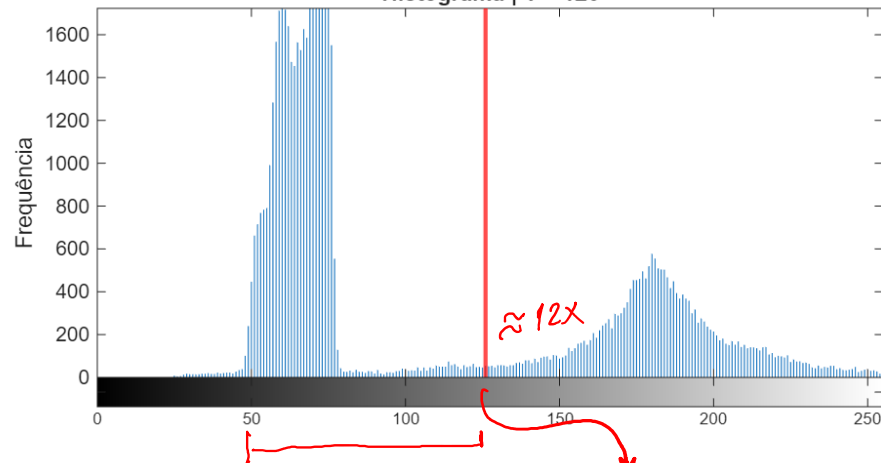
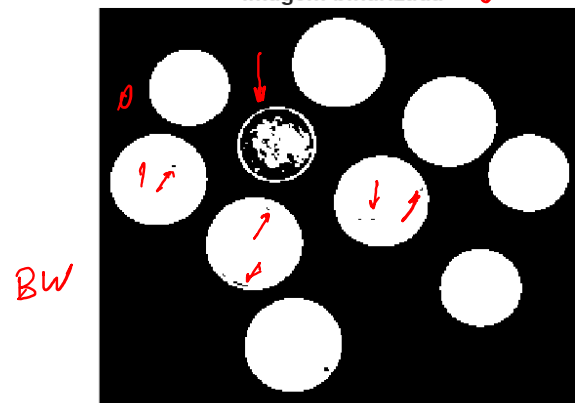
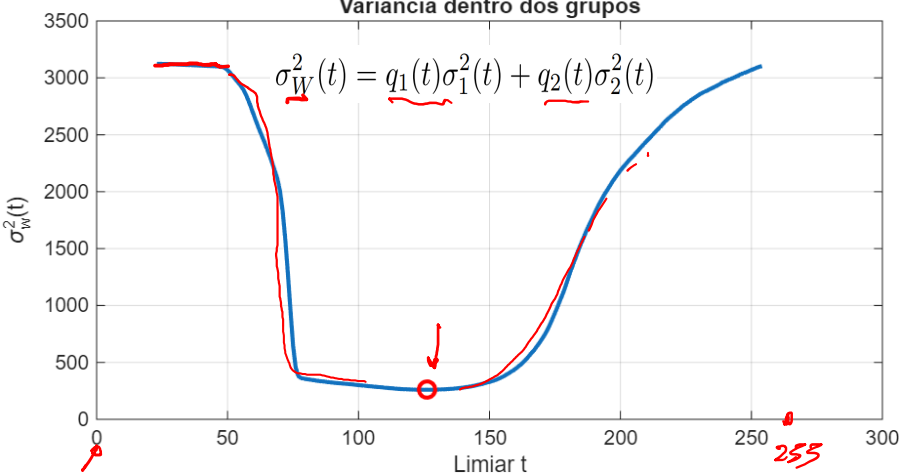


Imagem binarizada

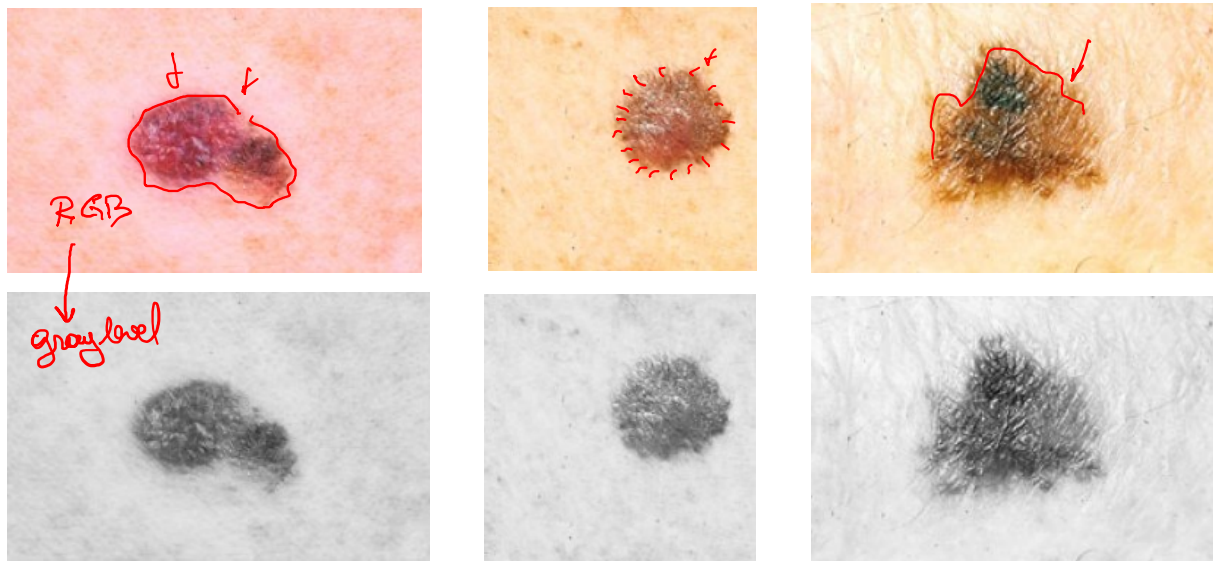


Variância dentro dos grupos



Limiarização global

- Usar Otsu para [tentar] segmentar automaticamente as imagens de lesões de pele.



Limiarização global

- Usar Otsu para [tentar] segmentar automaticamente as imagens de lesões de pele.

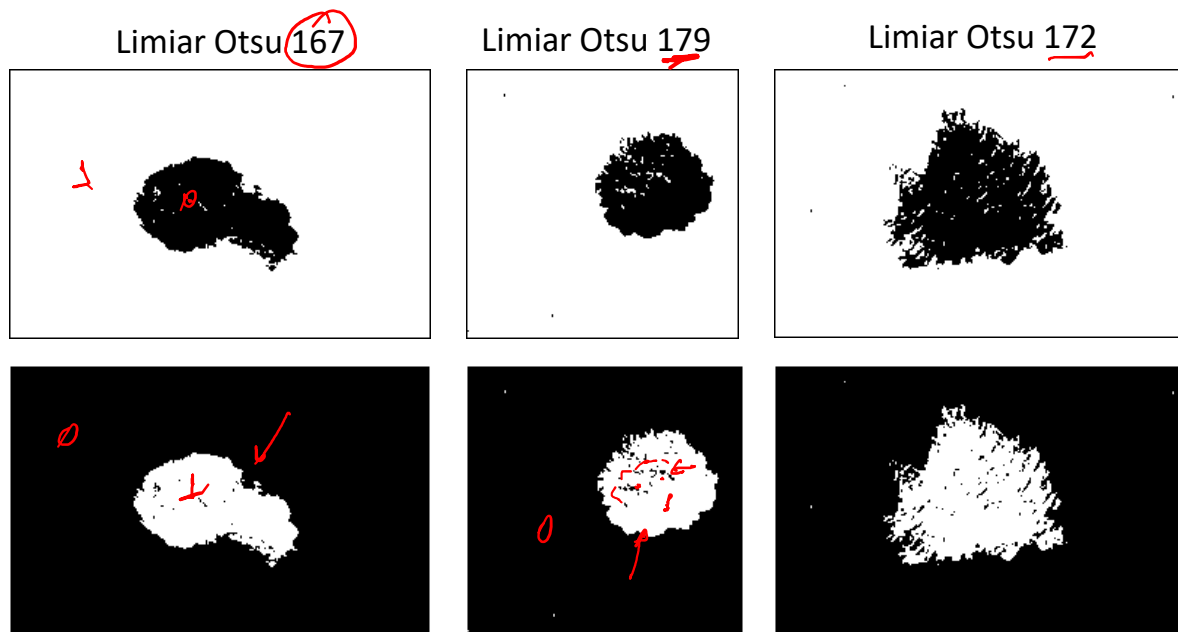


Image segmentation: Limiarização Adaptativa (thresholding)

Threshold adaptativo (LOCAL)

Em vez de usar um único limiar para toda a imagem, calcula-se um limiar local para cada região.

$$BW(x, y) = \begin{cases} 1, & I(x, y) > T(x, y) \\ 0, & I(x, y) \leq T(x, y) \end{cases}$$

Quando usar?

- Iluminação não uniforme
- Presença de sombras
- Fundo irregular
- Documentos, microscopia, imagens industriais

Algoritmo de limiarização adaptativa:

1. Definir uma vizinhança local ao redor de cada pixel
2. Calcular uma estatística local da vizinhança
Ex.: média, mediana ou média ponderada
3. Definir um limiar local T
4. Comparar o pixel com esse limiar
5. Gerar a imagem binária

Matlab:

```
BW = imbinarize(I, 'adaptive');
```

```
T = adapthresh(I, 0.5, 'NeighborhoodSize', N);
```

```
BW = imbinarize(I, T);
```

Por padrão o MATLAB trabalha com a média local ponderada (gaussian weighted)

O 0.5 é o parâmetro de sensibilidade (controlo o nível do limiar):
baixo (0.3) menos pixels=1, alto(0.7) mais pixels=1

$$T(x, y) \approx \mu(x, y) \cdot (1 - k)$$

Para uma imagem de $M \times N$:

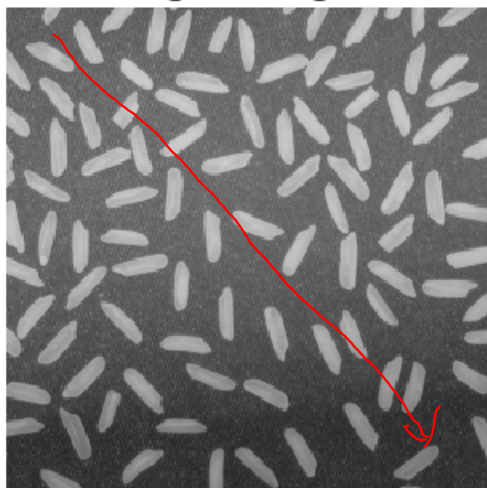
Nhood = 2 * floor([M N]/16) + 1; (valor padrão de limiar)

Imagem 256 x 256

NeighborhoodSize = [33 33]

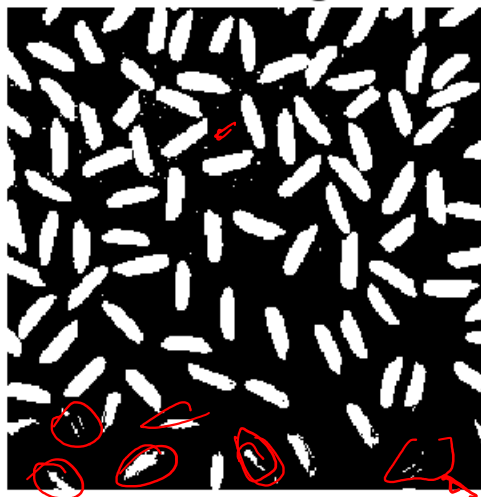
Comparação: Otsu vs Adaptativo

Imagem original



```
I = imread('rice.png');  
imshow(I)
```

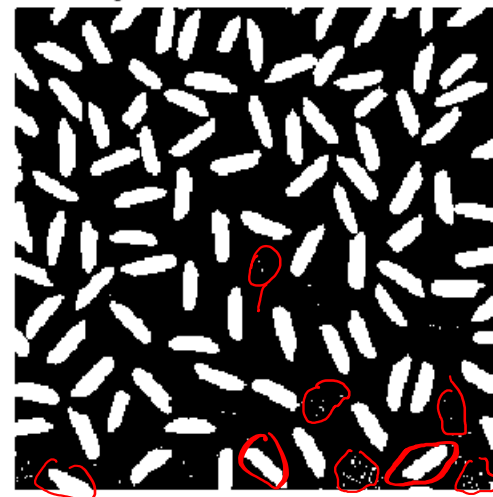
Otsu: limiar global



```
level = graythresh(I);  
BW_otsu = imbinarize(I, level);  
imshow(BW_otsu)
```

otsu

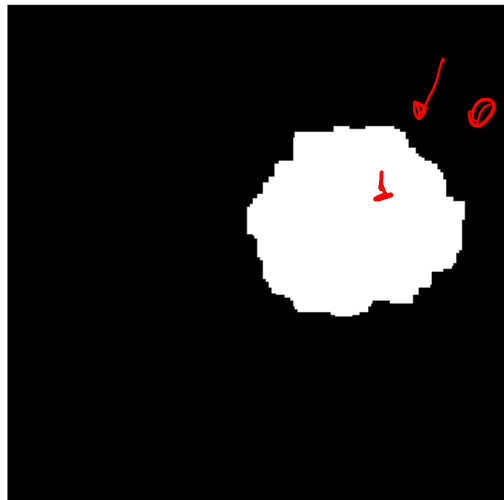
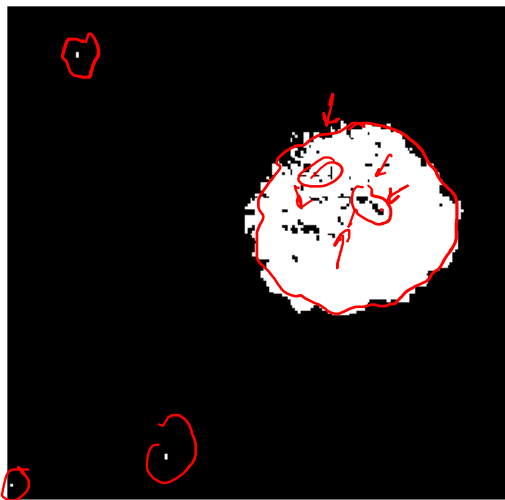
Adaptativo: limiar local



```
T = adaptthresh(I, 0.45);  
BW_adapt = imbinarize(I, T);  
imshow(BW_adapt)
```

Operações morfológicas

- Morfologia significa “estudo da forma, estrutura, aparência”. Então, as operações morfológicas tem o objetivo de analisar e manipular a forma (*shape*) dos objetos na imagem.



Operações morfológicas

- A base matemática da morfologia é a teoria de conjuntos.

Um conjunto de pixels binários denominado **elemento estruturante** (^{SE}structuring element) varre a imagem (outro conjunto de pixels binários) e as operações morfológicas são realizadas entre o elemento estruturante e os pixels da imagem contidos no elemento estruturante.

- Este processo de 'varrer a imagem com o elemento estruturante' é similar àquele que realizamos em um box filter, por exemplo. Mas ao invés de aplicar uma convolução, **aplicamos a operação morfológica entre máscara (elemento estruturante) e os pixels da imagem.** ^{SE}
- O **SE** pode ter um formato arbitrário e depende da aplicação.

Operações morfológicas

- **Imagens binárias**: Trataremos de **operações morfológicas** aplicadas a **imagens binárias** (BW).
 - Pixels 1 (branco): são os **pixels dos objetos**.
 - Pixels 0 (preto): são os **pixels do fundo**
- **Hot spot**: é o **ponto de referência** do elemento estruturante (SE). Este ponto de referência nem sempre é o **ponto central do elemento estruturante**.
- **Elemento estruturante**:
 - um ponto ativo tem valor '1';
 - um ponto não ativo (irrelevante) tem valor '0'.

Operações morfológicas

Vamos ver 5 operações morfológicas:

- Dilatação (delation) ✓
- Erosão (erosion) ✓
- Opening ✓
- Closing ✓
- Opening-Closing ✓

Operações morfológicas: Dilatação (dilation)

Dilatação (Dilation)

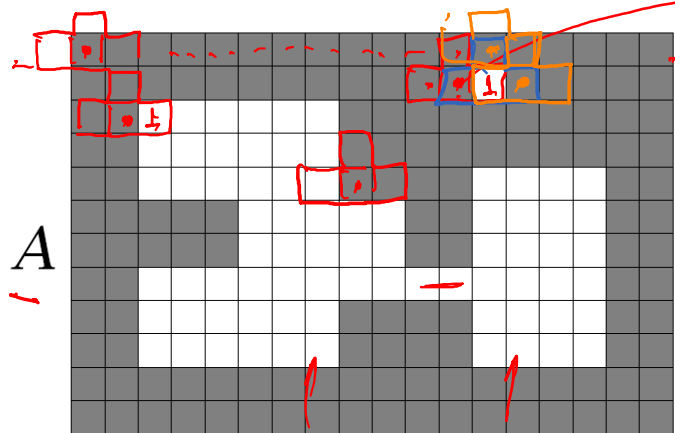
- “Engrossa” o objeto
- Expande regiões
- Preenche pequenos buracos

$$D = A \oplus SE$$

Procedimento da dilatação de **A** por um elemento estruturante **SE**:

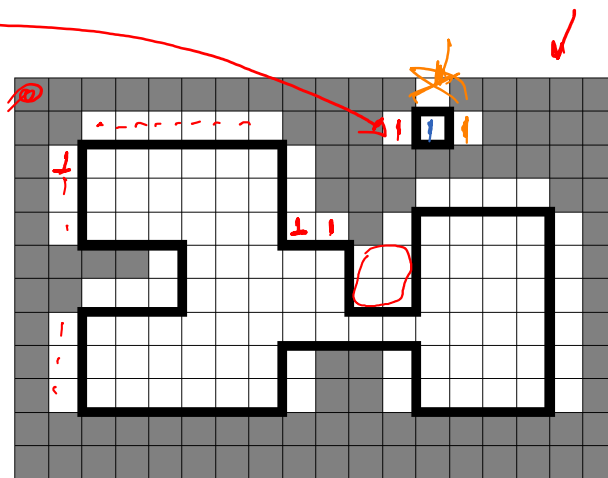
1. Posicione o **centro do SE** sobre um pixel
2. Observe todos os pixels cobertos pelo SE
3. Se pelo **menos um for 1** → o **pixel central** na saída vira 1

Dilatação funciona como um **OR local**



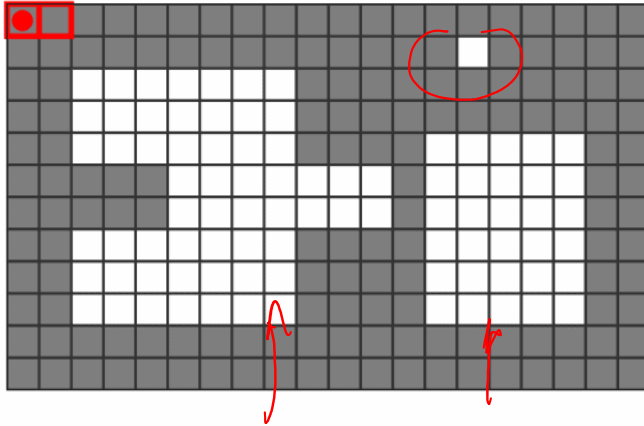
$$\oplus se \begin{array}{|c|} \hline \square \\ \hline \square \bullet \square \\ \hline \square \\ \hline \end{array} = D$$

SE

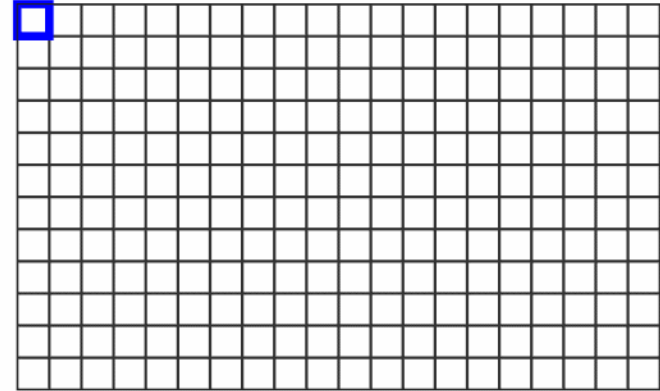


Dilatação morfológica: o SE varre a imagem

Imagem original A



Saída D = A $\oplus$ SE



Operações morfológicas: Erosão (erosion)

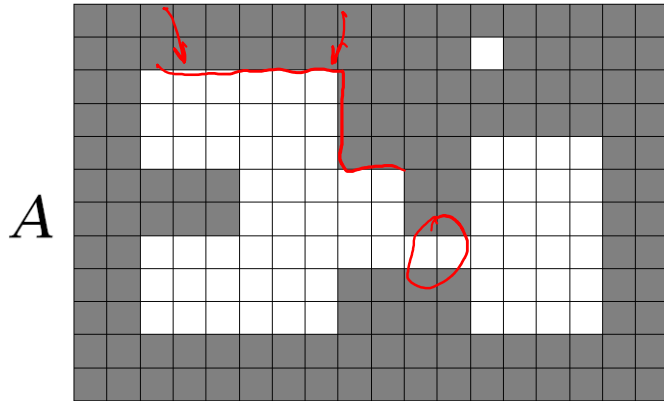
Erosão (Erosion)

- ✓ Encolhe os objetos
- ✓ Remove ruídos pequenos

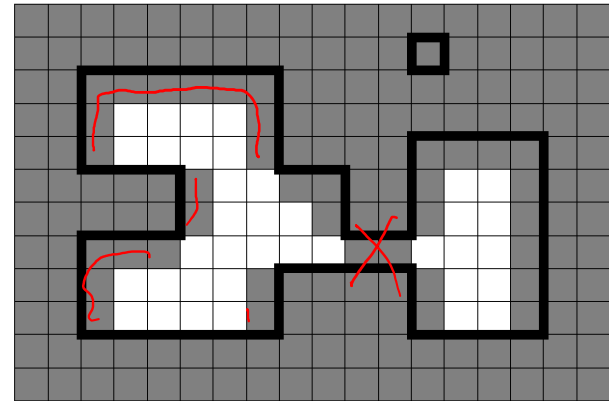
$$E = A \ominus se$$

Procedimento da Erosão de A por um elemento estruturante SE:

1. Posicione o **centro do SE** sobre um pixel
2. Observe todos os pixels cobertos pelo SE
3. Se **TODOS forem 1** → o pixel central na saída vira 1

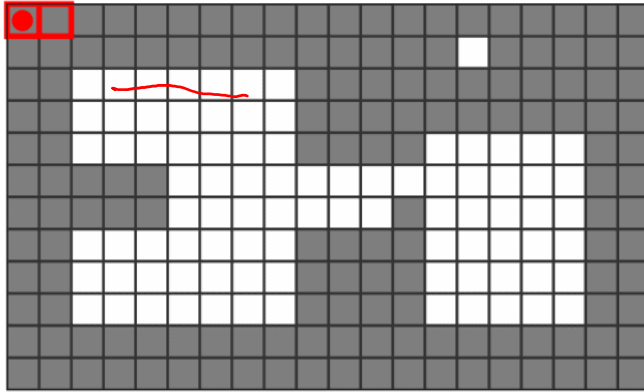


$$\ominus se \begin{array}{|c|} \hline \blacksquare \\ \hline \square \bullet \square \\ \hline \end{array} = E$$

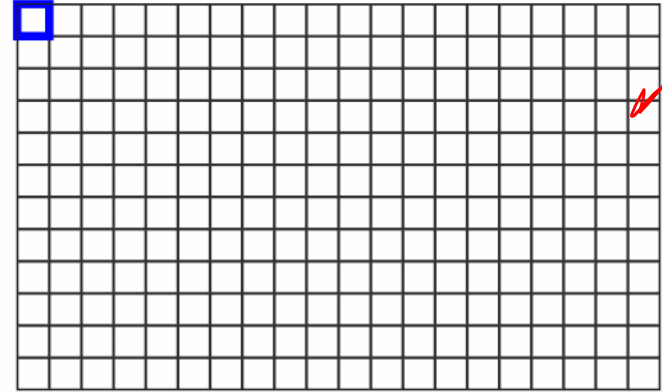


Erosão morfológica

Imagem original A



Saída E = A erosao SE



erosao_GIF_objetos_conectados.gif

Operações morfológicas: Abertura (Opening) → Limpa a imagem sem crescer os objetos

Definição: Erosão seguida de dilatação

Forma matemática: $A \circ SE = (A \ominus SE) \oplus SE$

Efeitos:

- Remove **estruturas pequenas**
- Elimina **ruído fino**
- Quebra **conexões finas**
- Suaviza contornos sem expandir significativamente o objeto

Procedimento:

Etapa 1 – Erosão

Todos os pixels cobertos pelo SE for 1 → saída 1

Caso contrário → 0

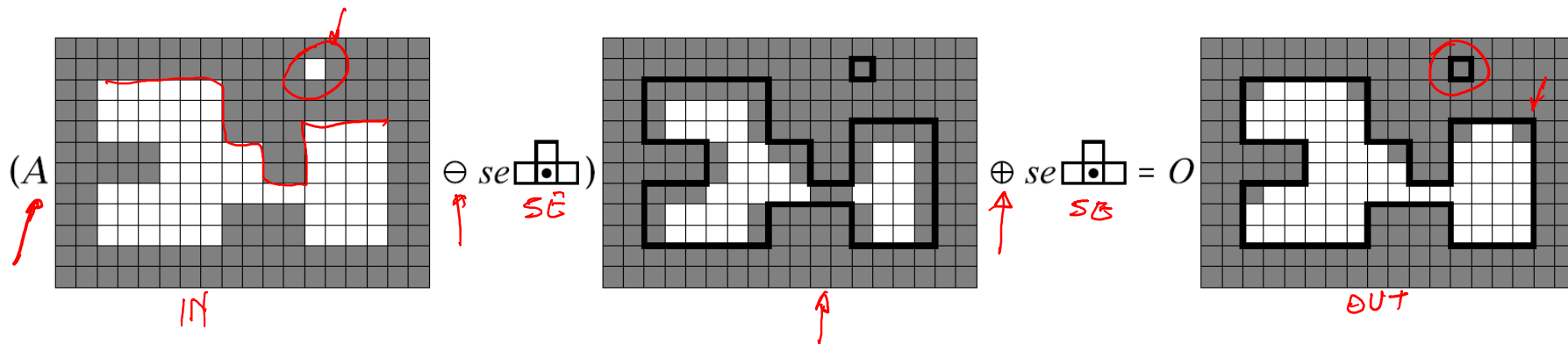
imagem encolhida

Etapa 2 — Dilatação

Pelo menos um pixel coberto pelo SE for 1 → saída 1

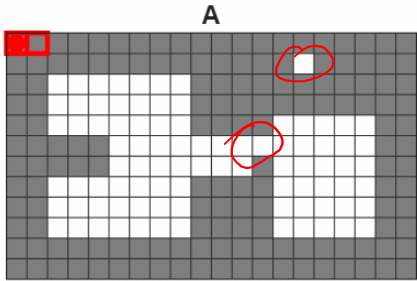
imagem volta a crescer

(sem recuperar o que foi perdido)



Operações morfológicas: **Abertura (Opening)** → **Limpa a imagem sem crescer os objetos**

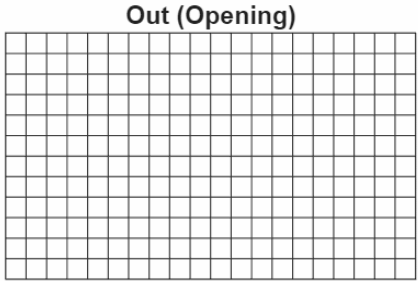
Opening: Etapa 1 (Erosão)



IN



INDILATAÇÃO



opening_GIF_SE.gif

Operações morfológicas: **Fechamento (Closing)** **Preenche buracos sem reduzir os objetos**

Definição: Dilatação seguida de erosão

Forma matemática: $A \cdot SE = (A \oplus SE) \ominus SE$

Efeitos:

- Preenhe buracos
- Conecta regiões próximas
- Suaviza contornos internos

Pequenos buracos são preenchidos e conexões são reforçadas

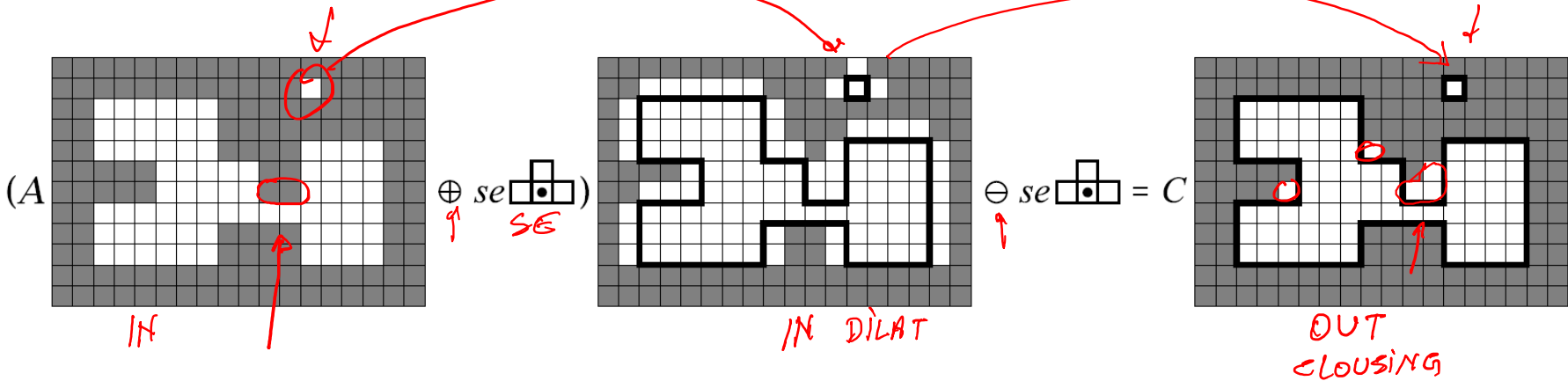
Procedimento:

Etapa 1 — Dilatação

Pelo menos um pixel coberto pelo SE for 1 → saída 1
imagem expande

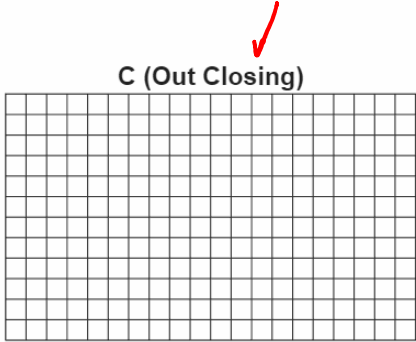
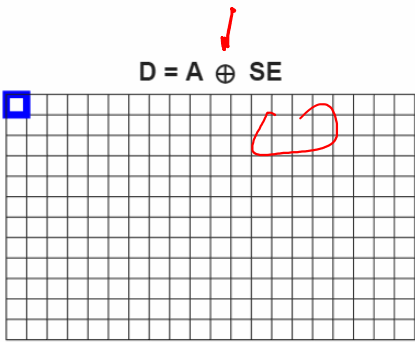
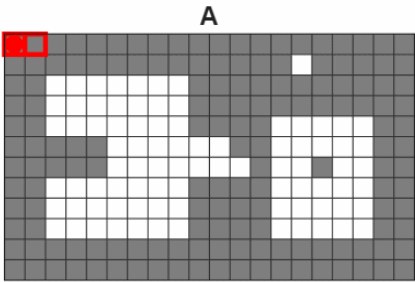
Etapa 2 – Erosão

Todos os pixels cobertos pelo SE forem 1 → saída 1
Caso contrário → 0
imagem encolhe



Operações morfológicas: **Fechamento (closing)** **Preenche buracos sem reduzir os objetos**

Closing: Etapa 1 (Dilatação)



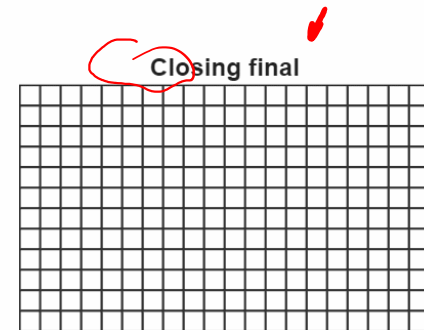
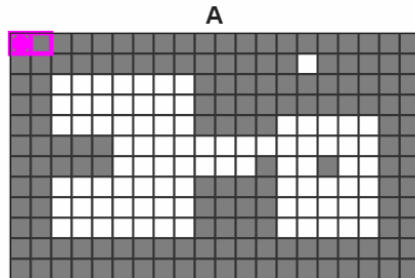
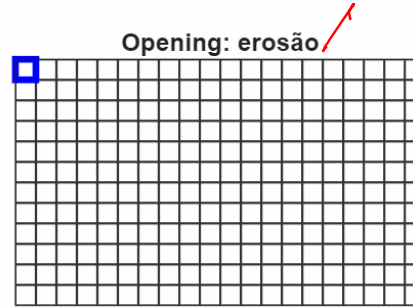
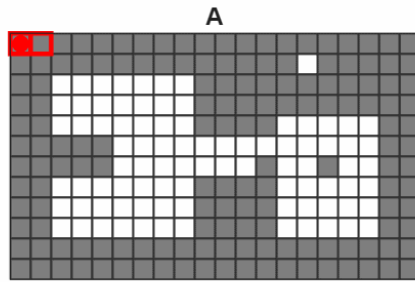
closing_GIF.gif

Operações morfológicas: **Comparação das operações vistas**

Operação	Efeito
Dilatação 	cresce 
Erosão 	reduz 
Abertura 	limpa 
Fechamento 	preenche 

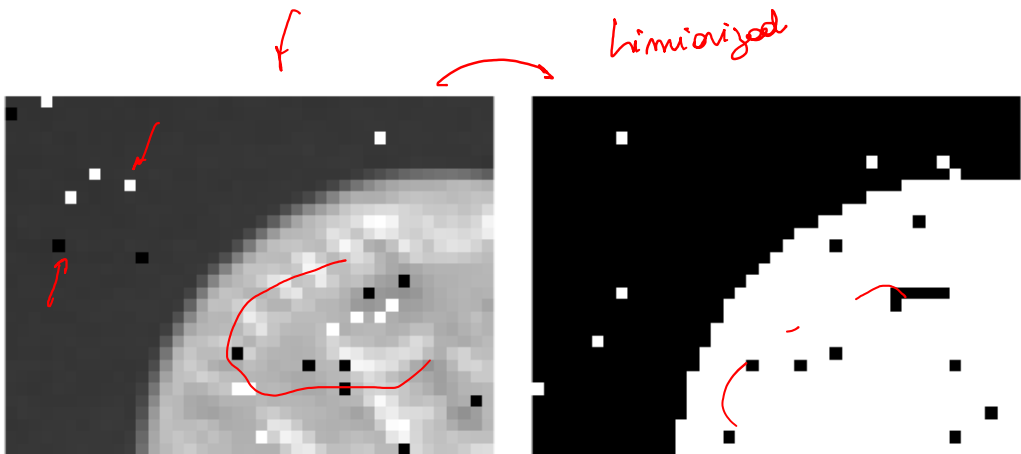
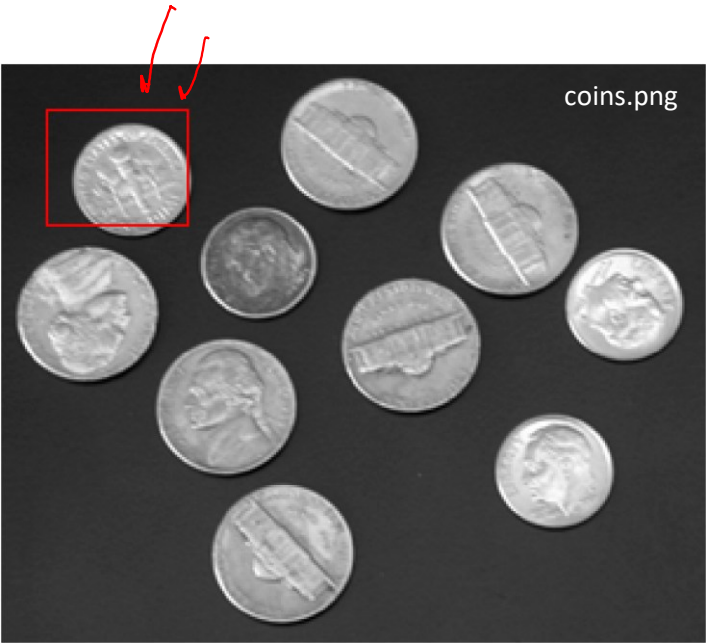
Operações morfológicas: Opening vs Closing

Comparação animada: Opening vs Closing — Etapa 1



opening_vs_closing_animado.gif

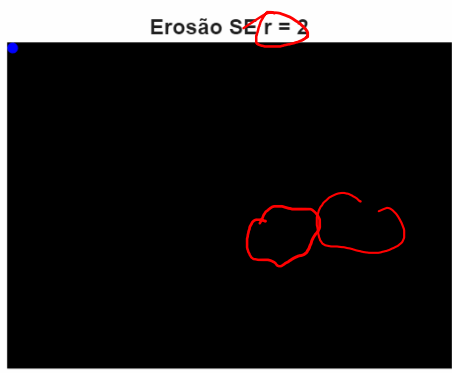
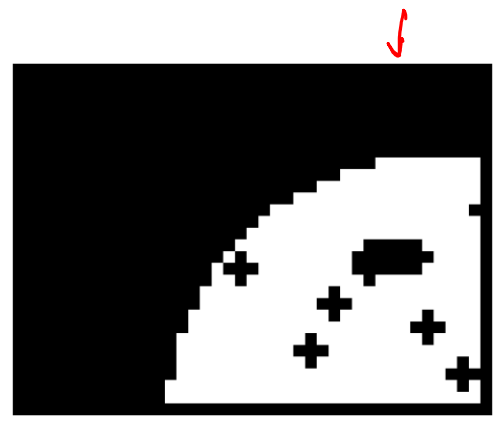
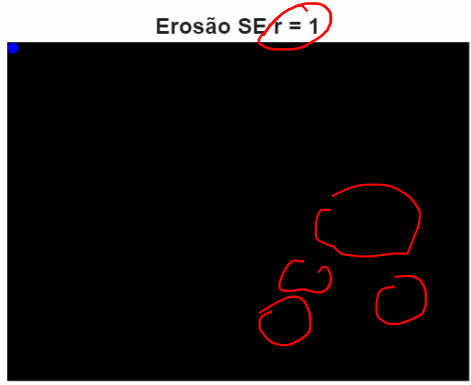
Operações morfológicas: Erosão (erosion) – Exemplo prático



```
I_noisy = imnoise(Igray, 'salt & pepper', 0.02);  
BW = imbinarize(I_noisy, 0.6);
```

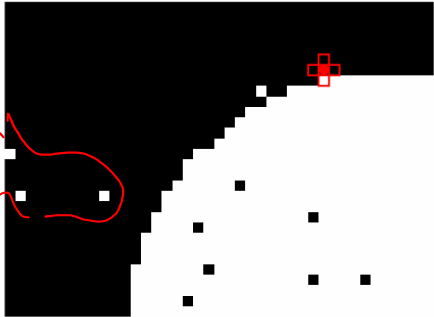
```
Igray = Igray(20:floor(M/5), 20:floor(N/5));  
rectangle('Position',[20 20 floor(N/5) floor(M/5)], 'EdgeColor','r', 'LineWidth',2);
```

Operações morfológicas: Erosão (erosion) – Exemplo prático

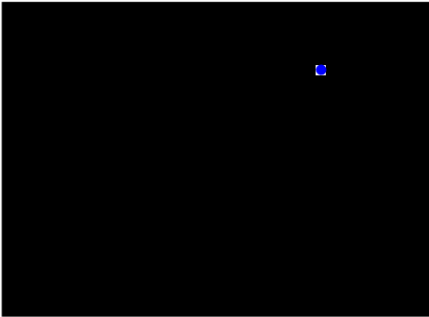


Operações morfológicas: Dilatação (Dilation) – Exemplo prático

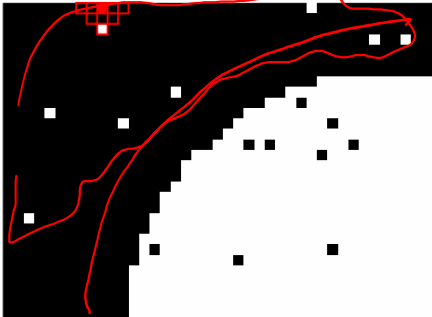
Moedas binarizadas + ruído S&P



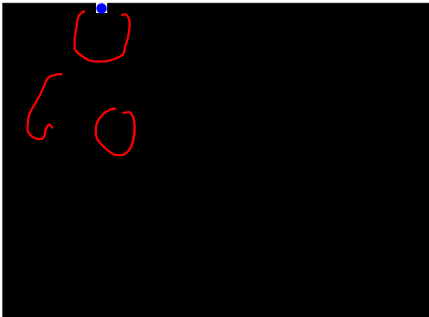
Dilation SE r = 1



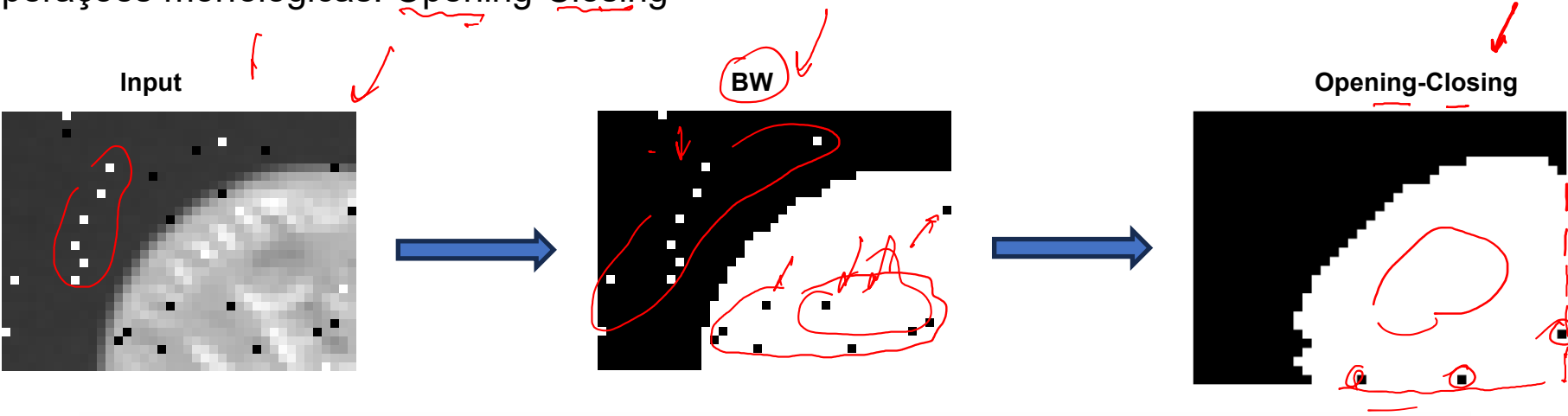
Moedas binarizadas + ruído S&P



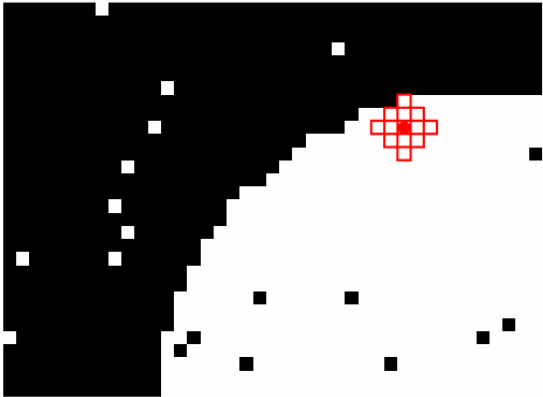
Dilation SE r = 2



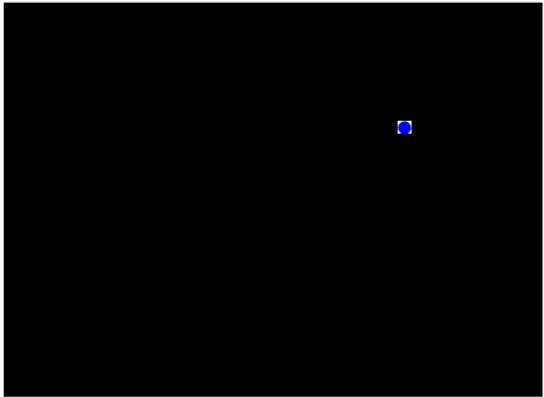
Operações morfológicas: Opening-Closing



Input Image for Opening/Closing



Opening (Erosion)



Operações morfológicas

Original



Limiarizada (binária)



Dilatação



Erosão



Fechamento



Abertura



Fechamento: funde
quebras estreitas,
elimina pequenos
buracos, preenche
espaços de contornos.

Abertura: quebra
ligações estreitas,
elimina protrusões
finas.

Operações morfológicas: Open-close filter

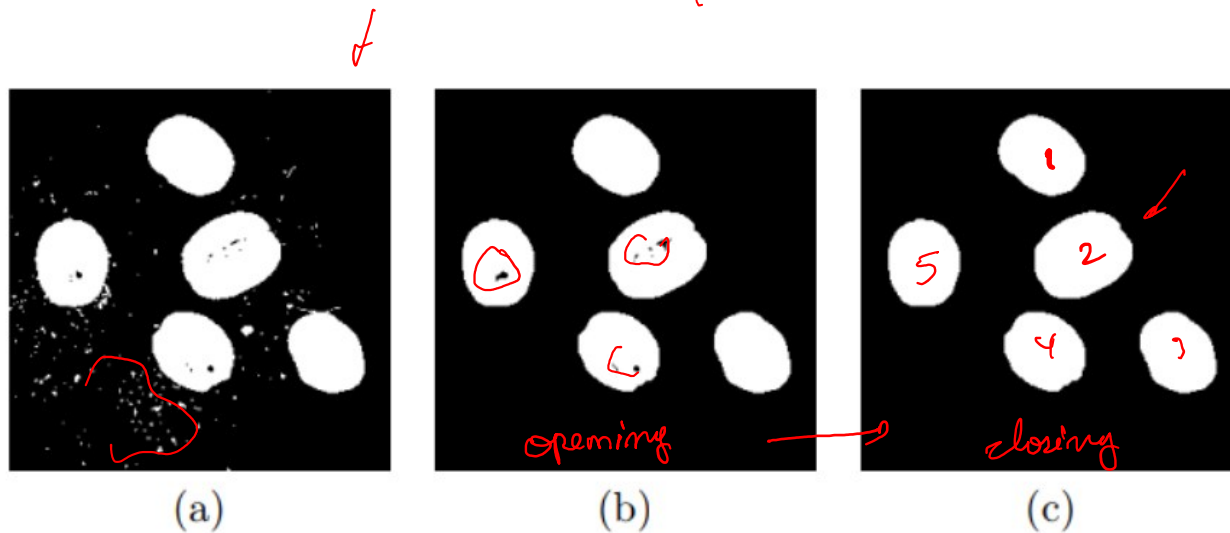


Figure 24.5: Open-close filtering using a SE of disk shape with radius $r = 5$. (a) Input image; (b) opening; (c) subsequent closing.

Existem muitos tipos de operações morfológicas...

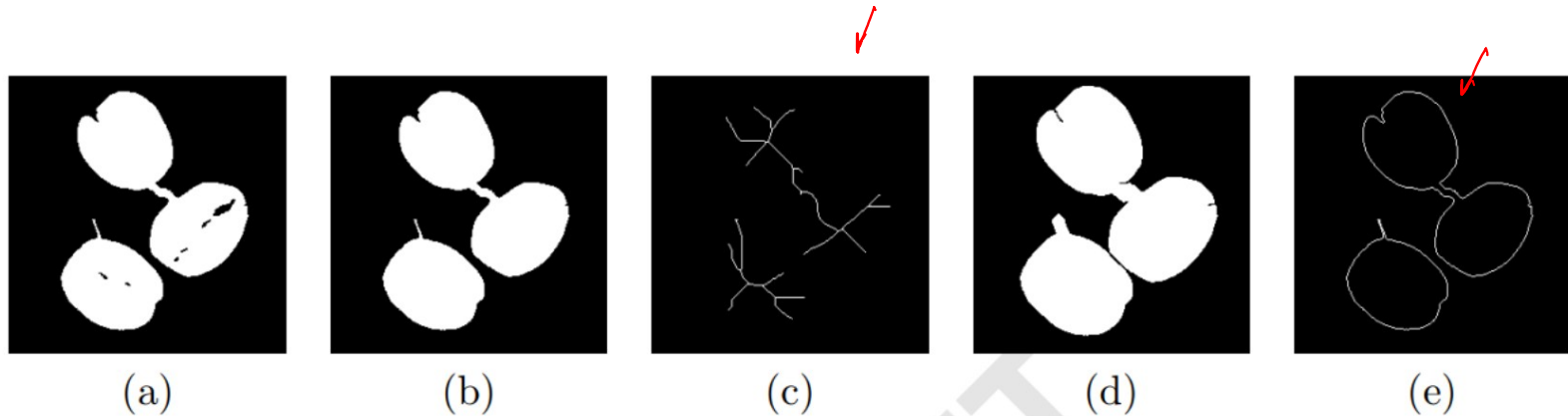


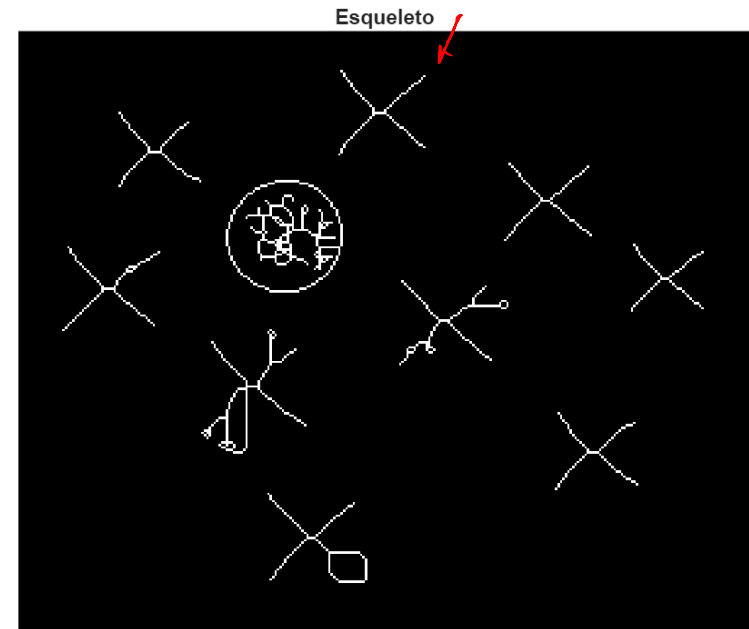
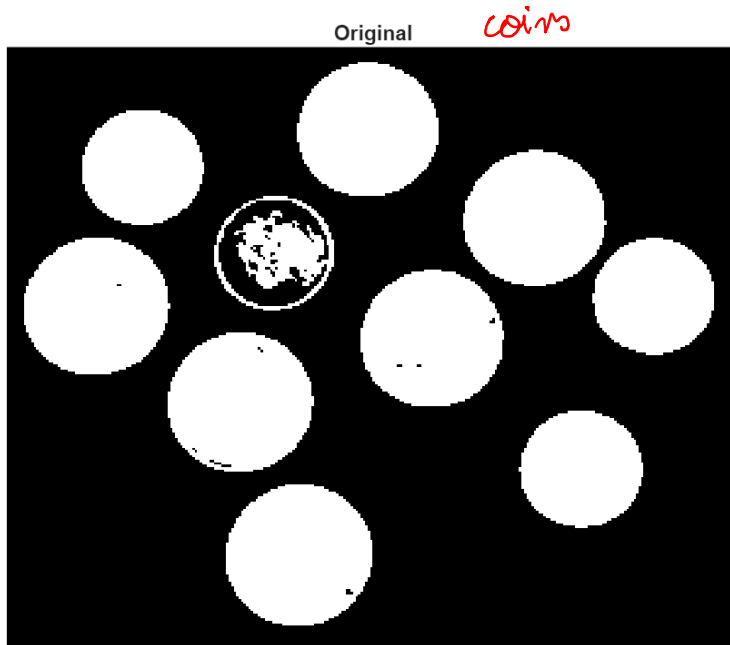
Figure 24.7: Morphological techniques obtained with `imfill`, `bwmorph` and `bwperim` functions. (a) Original image, (b) original image with filled holes, (c) skeleton of (b), (d) thickening of (b), (e) perimeter extraction of (b).

```
BW_skel = bwmorph(BW, 'skel', Inf);
```

Esqueletização (skeleton)

Esqueletização reduz objetos à sua estrutura central preservando a conectividade, mas sensível a ruído

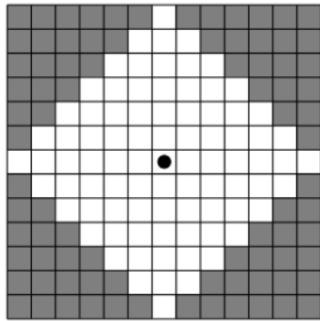
O algoritmo remove iterativamente pixels das bordas preservando a conectividade.”



```
clc, clear, close all
BW = imbinarize(imread('coins.png'));
skel = bwmorph(BW, 'skel', Inf);
figure
subplot(1,2,1), imshow(BW), title('Original')
subplot(1,2,2), imshow(skel), title('Esqueleto')
```

O elemento estruturante sempre é configurável

```
r = 6;  
se = strel('diamond',r);  
se.Neighborhood
```

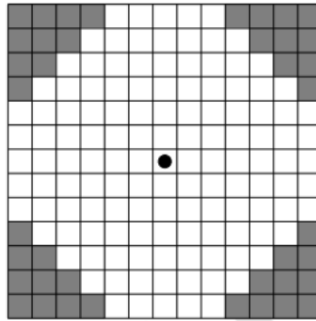


```
len = 7; deg = 0;  
se = strel('line',len,deg);  
se.Neighborhood
```

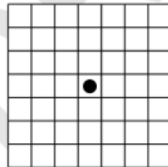


p

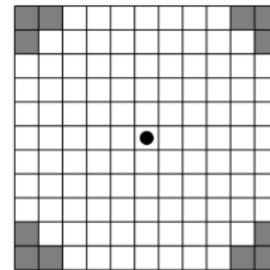
```
r = 6;  
se = strel('octagon',r);  
se.Neighborhood
```



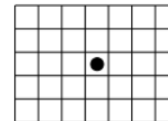
```
w = 7;  
se = strel('square',w);  
se.Neighborhood
```



```
r = 6;  
se = strel('disk',r);  
se.Neighborhood
```



```
m = 5; n = 7;  
se = strel('rectangle',[m n]);  
se.Neighborhood
```



21:10
↓
rebinommo
21:25

Rotulação de regiões binárias (labeling)

- Quantas lesões de pele tem na imagem?
- Quantas células tem na imagem?



Rotulação de regiões binárias (labeling)

```
clear all, close all
clc

bw = [ 0 0 0 0 0 0 0 0 0 0
       0 0 0 0 0 0 0 0 1 0
       0 0 1 1 1 0 0 1 0 0
       0 0 1 1 1 0 1 1 1 0
       0 0 0 0 0 1 1 1 0 0
       0 0 0 0 0 0 1 1 1 0
       0 0 0 1 0 0 0 1 1 0
       0 0 1 1 0 0 1 1 0 0
       0 1 1 1 0 1 1 0 0 0
       0 0 0 0 0 0 0 0 0 0];
```

```
bw = logical(bw);
```

```
% A saída de bwlabel
% é da classe double
[rv4 nrv4] = bwlabel(bw, 4);
[rv8 nrv8] = bwlabel(bw, 8);
```

```
% Aplica pseudocores na
% imagem rotulada para
% visualização
rv4rgb = label2rgb(rv4);
rv8rgb = label2rgb(rv8);
disp(['n. obj em rv4 = ', ...
      num2str(nrv4)])
disp(['n. obj em rv8 = ', ...
      num2str(nrv8)])
```

```
% Display
figure, image(rv4rgb)
title('rv4rgb')
figure, image(rv8rgb)
title('rv8rgb')
```

```
>> rv4
```

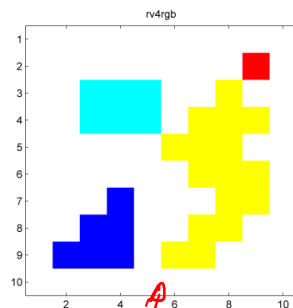
```
rv4 =
```

```
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 4 0
0 0 2 2 2 0 0 0 0 0
0 0 2 2 2 0 3 3 3 0
0 0 0 0 0 0 3 3 3 0
0 0 0 0 0 0 0 3 3 0
0 0 0 0 1 0 0 1 3 3 0
0 0 1 1 1 0 0 3 3 0
0 1 1 1 1 0 3 3 0 0
0 0 0 0 0 0 0 0 0 0
```

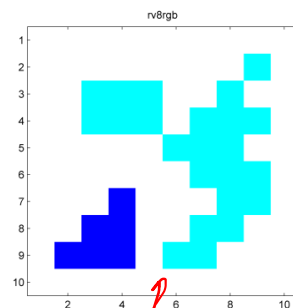
```
>> rv8
```

```
rv8 =
```

```
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 2 0
0 0 0 2 2 2 0 0 2 0
0 0 0 2 2 2 0 2 2 0
0 0 0 0 0 0 2 2 2 0
0 0 0 0 0 0 0 2 2 0
0 0 1 1 1 0 0 2 2 0
0 1 1 1 1 0 2 2 0 0
0 0 0 0 0 0 0 0 0 0
```

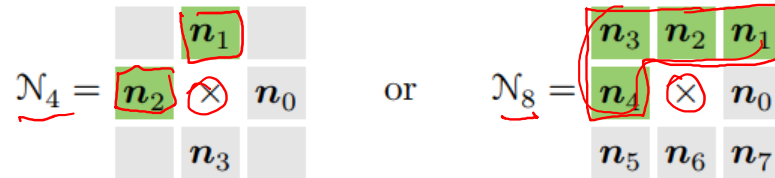


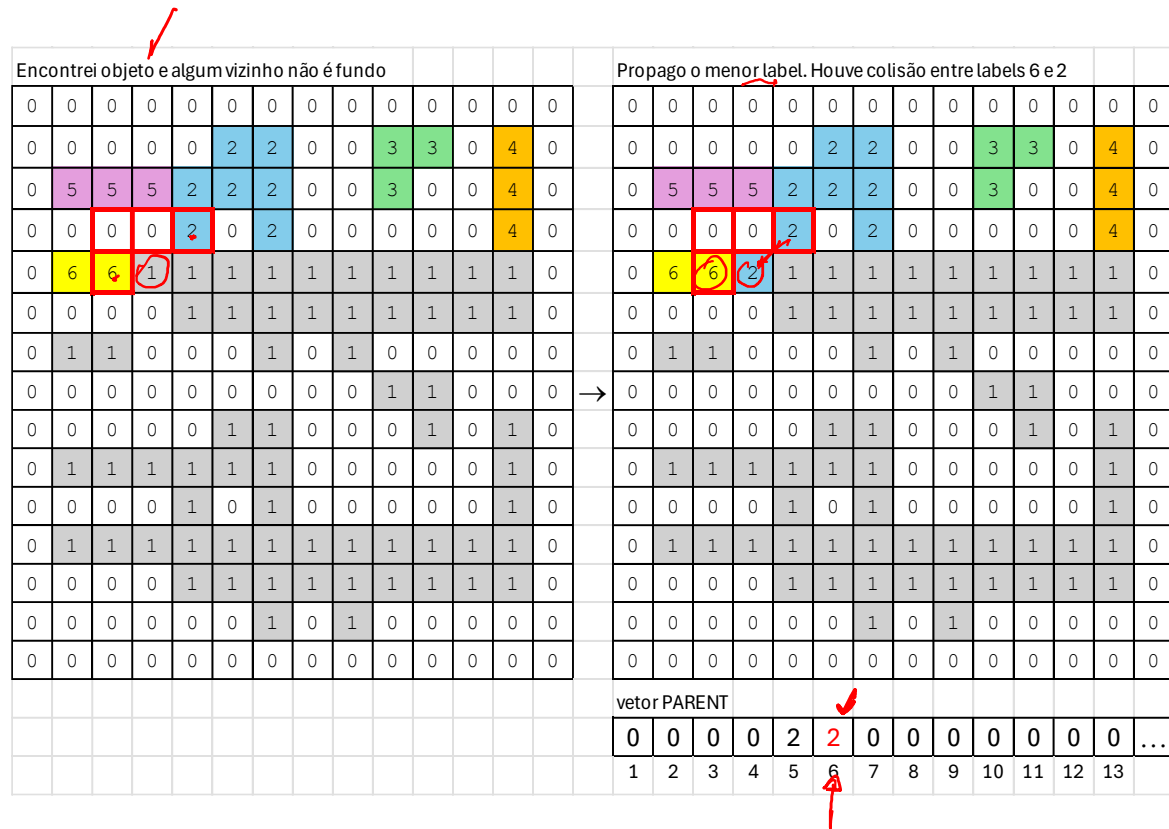
4 obj



2 obj

Rotulação de regiões binárias (labeling)

[illegible]



Propago o menor label. Houve colisão entre labels 4 e 2

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	3	0	4	0
0	5	5	5	2	2	2	0	0	3	0	0	4	0
0	0	0	0	2	0	2	0	0	0	0	0	4	0
0	6	6	2	2	2	2	2	2	2	2	2	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	1	1	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	1	1	0	0	0	1	0	1	0
0	1	1	1	1	1	1	0	0	0	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0
vetor PARENT													
0	0	0	2	2	2	0	0	0	0	0	0	0	...
1	2	3	4	5	6	7	8	9	10	11	12	13	

Propago o menor label. Houve colisão entre labels 10 e 8

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	3	0	4	0
0	5	5	5	2	2	2	0	0	3	0	0	4	0
0	0	0	0	2	0	2	0	0	0	0	0	4	0
0	6	6	2	2	2	2	2	2	2	2	2	2	0
0	0	0	0	2	2	2	2	2	2	2	2	2	0
0	7	7	0	0	0	2	0	2	0	0	0	0	0
0	0	0	0	0	0	0	0	0	2	2	0	0	0
0	0	0	0	0	8	8	0	0	0	2	0	9	0
0	10	10	10	8	8	8	0	0	0	0	0	9	0
0	0	0	0	8	8	0	1	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0
vetor PARENT													
0	0	0	2	2	2	0	0	0	8	0	0	0	...
1	2	3	4	5	6	7	8	9	10	11	12	13	

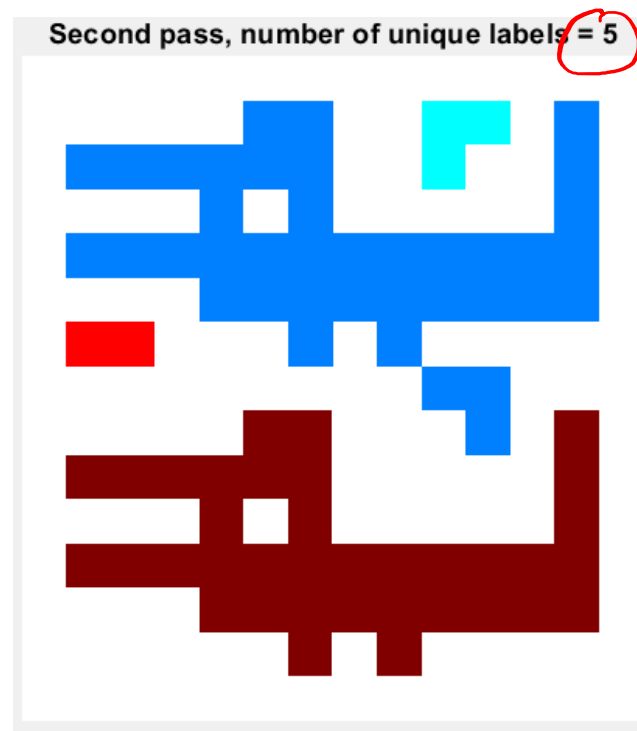
Propago o menor label. Houve colisão entre labels 11 e 8																	
0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0	0	0	0	0	2	2	0	0	3	3	0	4	0				
0	5	5	5	2	2	2	0	0	3	0	0	4	0				
0	0	0	0	2	0	2	0	0	0	0	0	4	0				
0	6	6	2	2	2	2	2	2	2	2	2	2	0				
0	0	0	0	2	2	2	2	2	2	2	2	2	0				
0	7	7	0	0	0	2	0	2	0	0	0	0	0				
0	0	0	0	0	0	0	0	0	2	2	0	0	0				
0	0	0	0	0	8	8	0	0	0	2	0	9	0				
0	10	10	10	8	8	8	0	0	0	0	0	9	0				
0	0	0	0	8	0	8	0	0	0	0	0	9	0				
0	11	11	8	1	1	1	1	1	1	1	1	1	0				
0	0	0	0	1	1	1	1	1	1	1	1	1	0				
0	0	0	0	0	0	1	0	1	0	0	0	0	0				
0	0	0	0	0	0	0	0	0	0	0	0	0	0				
vetor PARENT																	
0	0	0	2	2	2	0	0	0	8	8	0	0	...				
1	2	3	4	5	6	7	8	9	10	11	12	13					

Propago o menor label. Houve colisão entre labels 9 e 8																	
0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0	0	0	0	0	2	2	0	0	3	3	0	4	0				
0	5	5	5	2	2	2	0	0	3	0	0	4	0				
0	0	0	0	2	0	2	0	0	0	0	0	4	0				
0	6	6	2	2	2	2	2	2	2	2	2	2	0				
0	0	0	0	2	2	2	2	2	2	2	2	2	0				
0	7	7	0	0	0	2	0	2	0	0	0	0	0				
0	0	0	0	0	0	0	0	0	2	2	0	0	0				
0	0	0	0	0	8	8	0	0	0	2	0	9	0				
0	10	10	10	8	8	8	0	0	0	0	0	9	0				
0	0	0	0	8	0	8	0	0	0	0	0	9	0				
0	11	11	8	8	8	8	8	8	8	8	8	8	1				
0	0	0	0	1	1	1	1	1	1	1	1	1	0				
0	0	0	0	0	0	1	0	1	0	0	0	0	0				
0	0	0	0	0	0	0	0	0	0	0	0	0	0				
vetor PARENT																	
0	0	0	2	2	2	0	0	0	8	8	8	8	0	0	...		
1	2	3	4	5	6	7	8	9	10	11	12	13					

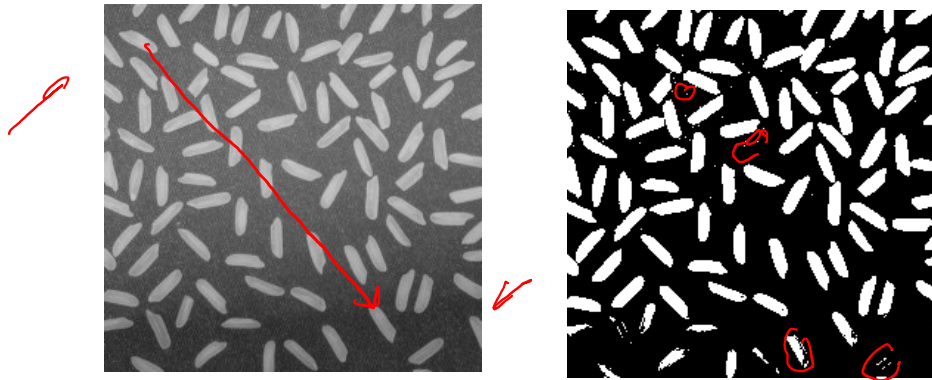
Fim do pass 1 (union ou label propagation)													
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	3	0	4	0
0	5	5	5	2	2	2	0	0	3	0	0	4	0
0	0	0	0	2	0	2	0	0	0	0	0	4	0
0	6	6	2	2	2	2	2	2	2	2	2	2	0
0	0	0	0	2	2	2	2	2	2	2	2	2	0
0	7	7	0	0	0	2	0	2	0	0	0	0	0
0	0	0	0	0	0	0	0	0	2	2	0	0	0
0	0	0	0	0	8	8	0	0	0	2	0	9	0
0	10	10	10	8	8	8	0	0	0	0	0	9	0
0	0	0	0	8	0	8	0	0	0	0	0	9	0
0	11	11	8	8	8	8	8	8	8	8	8	8	0
0	0	0	0	8	8	8	8	8	8	8	8	8	0
0	0	0	0	0	0	0	8	0	8	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0
vetor PARENT													
0	0	0	2	2	2	0	0	8	8	8	0	0	...
1	2	3	4	5	6	7	8	9	10	11	12	13	



- Implementação do deste algoritmo disponível no material de aula, arquivo *unionfind_labeling_v251.m*



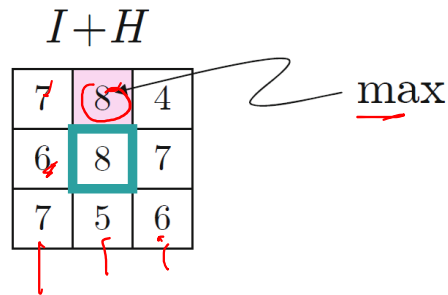
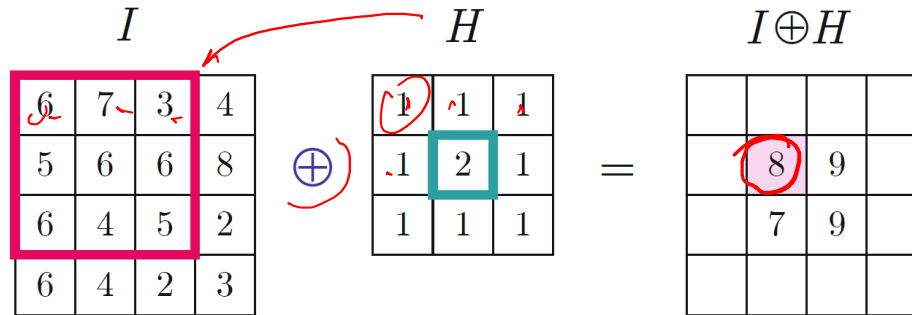
- A segmentação por limiarização global pode ser comprometida em situações nas quais a iluminação na imagem não é uniforme (homogênea):



- Existem diferentes abordagens para a solução. Por exemplo:
 - Transformada top-hat
 - Limiarização local (adaptativa)

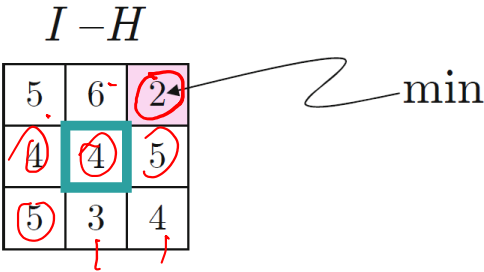
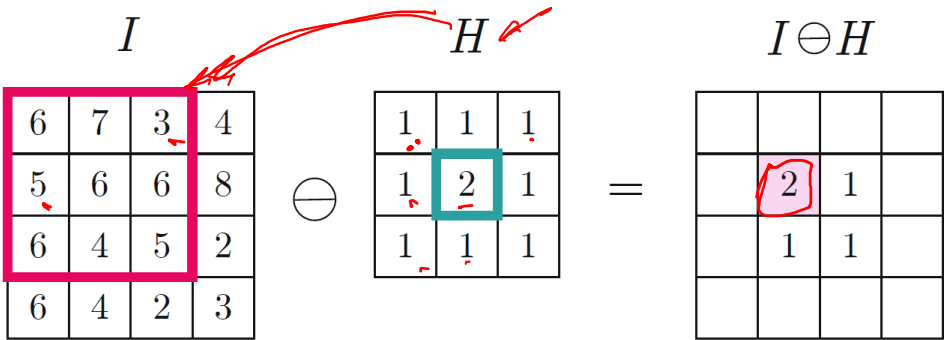
Dilatação grayscale

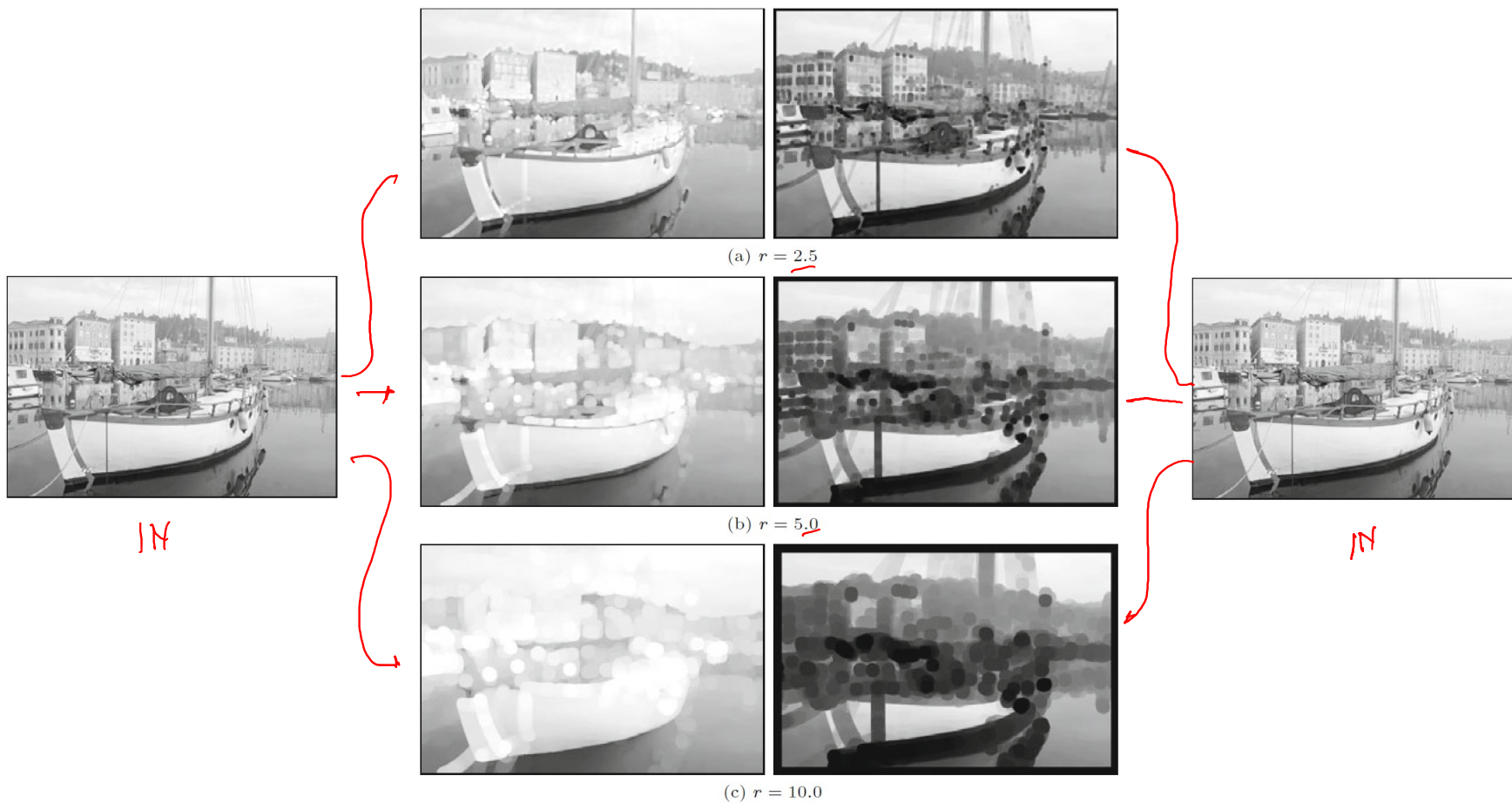
$$(I \oplus H)(u, v) = \max_{(i,j) \in H} (I(u+i, v+j) + H(i, j))$$



Erosão grayscale

$$(I \ominus H)(u, v) = \min_{(i, j) \in H} (I(u+i, v+j) - \overline{H(i, j)})$$





Transformada top-hat: Correção de iluminação

- A transformada top-hat pode ser utilizada como uma etapa de pré-processamento da imagem, para uniformizar a iluminação do fundo (background), antes de aplicar a limiarização global.
- A transformada top-hat TopHat de uma imagem grayscale G é:

$$\text{TopHat} = G - (G \circ S)$$

Handwritten red notes: a checkmark under the minus sign, and a red arrow pointing to the expression (G ∘ S) with the text "abertura" written next to it.

onde '∘' é a operação morfológica de abertura de G por um elemento estruturante S e '-' é uma subtração de imagens.

- Assim como nas operações morfológicas binárias, a abertura morfológica de uma imagem grayscale (**G ∘ S**) é o resultado da erosão seguida da dilatação, isto é, aplicar a erosão na imagem G usando S e depois aplicar a dilatação na imagem erodida usando S.

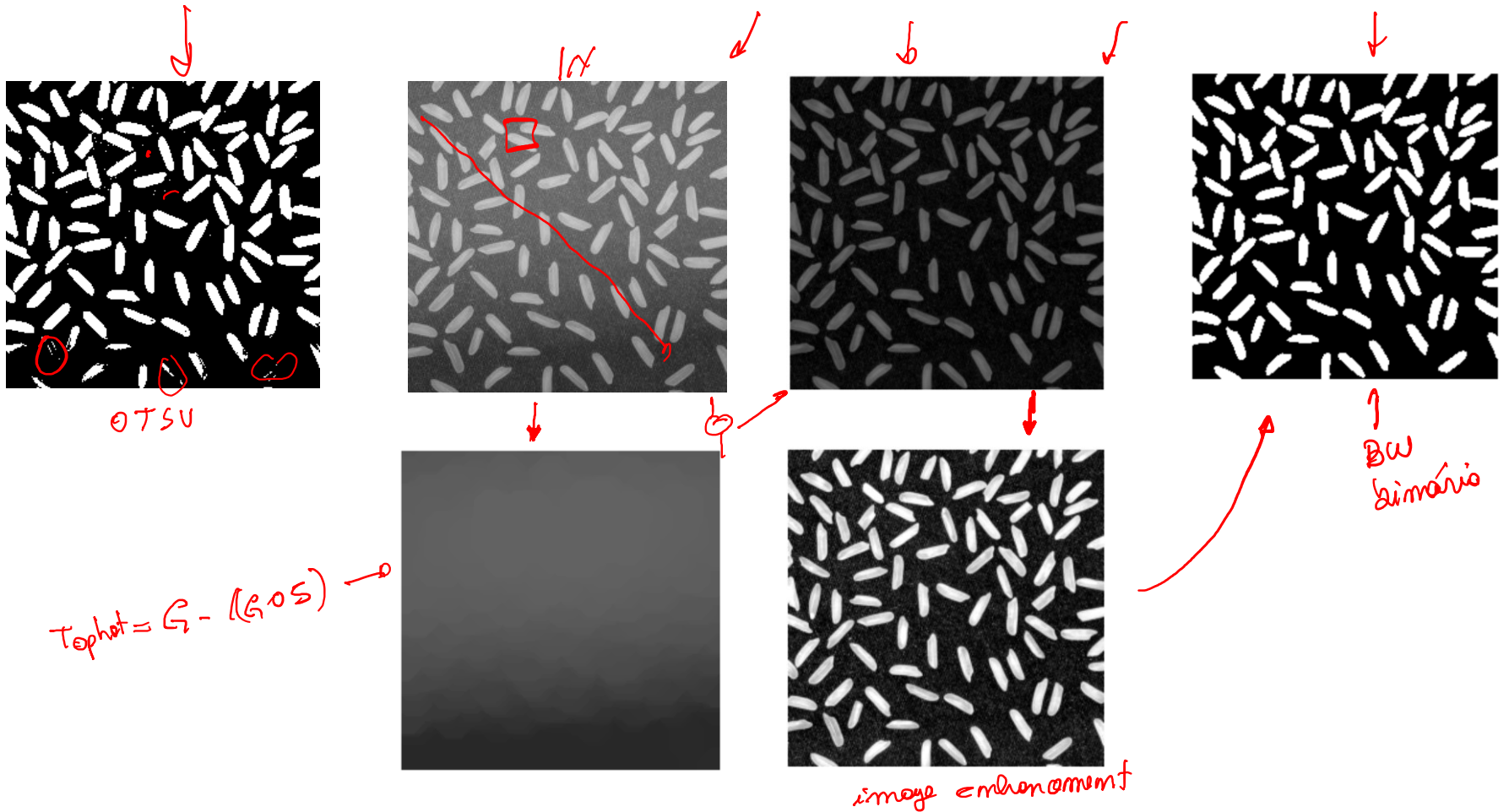
Ver:

<https://www.mathworks.com/help/images/correcting-nonuniform-illumination.html>

Handwritten red arrow pointing to the URL.

Transformada top-hat

<https://www.mathworks.com/help/images/correcting-nonuniform-illumination.html>



Th local algoritmo 1

<https://www.mathworks.com/help/images/ref/adaptthresh.html>

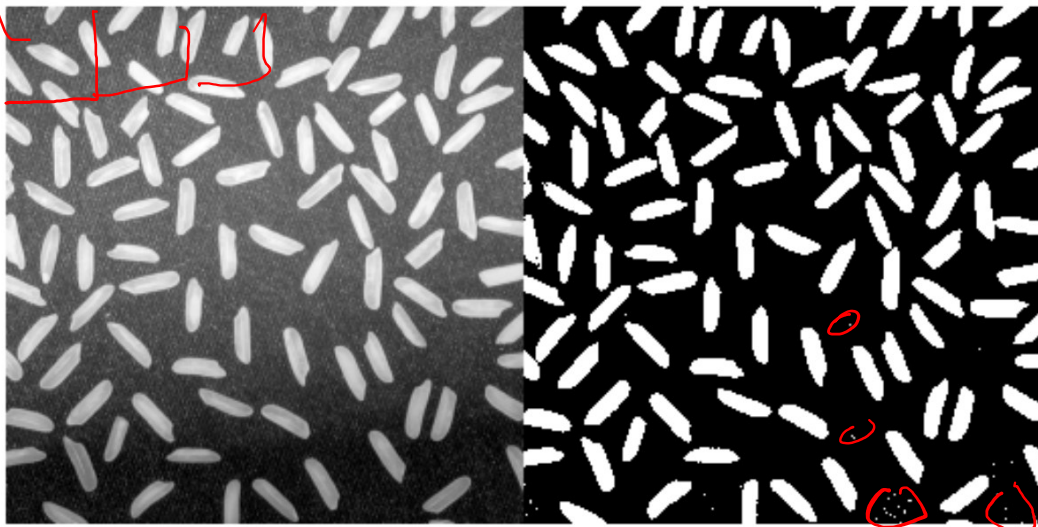
calcular, para cada pixel, a média em uma vizinhança $s \times s$ e realizar seguinte operação:

- se o valor do pixel é t por cento menor que a média, ele é considerado '0'
- caso contrário, é considerado '1'

$$\begin{aligned} f(i, j) &< (1 - t) \cdot \mu(i, j) \Rightarrow 0 \\ f(i, j) &> (1 - t) \cdot \mu(i, j) \Rightarrow 1 \end{aligned}$$

$\mu(i, j)$ é a média da região $S \times S$

Ou seja, o limiar não é fixo — ele é proporcional à **iluminação local**.



Isso é basicamente o que o **adaptthresh()** faz:

```
T = adaptthresh(I, 0.5); % sensibilidade  
BW = imbinarize(I, T);
```

Th local algoritmo 2

```
% adaptive Th [script]
% Tutorial 'Adaptive Thresholding',
% capítulo 15, páginas 350 até 352 do livro
% Oge Marques, Practical image and video
% processing using MATLAB, Wiley, 2011.
clear all; close all; clc

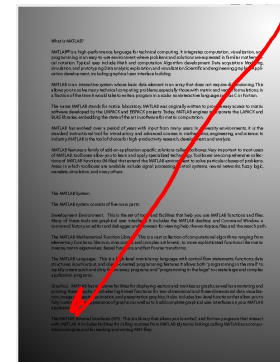
g = imread('gradient with text.tif');
figure, imshow(g), title('in')

% Global Th
ggth = im2bw(g, graythresh(g));
figure, imshow(ggth), title('ggth')
figure, imhist(g) % otsu fail

% Adaptive Th
% Função blockproc é sem overlap
% Cria function handle
fun = @(myBlock) adaptThStd(myBlock.data);
gath = blockproc(g, [10 10], fun); % processa
figure, imshow(gath), title(gath)
```

```
function y = adaptThStd(x)
% x: bloco da imagem (subimagem)
% y: bloco processado

% std baixo -> é fundo
if(std2(x)) < 1
    % Sai um bloco de uns
    y = ones(size(x,1),size(x,2));
% std alto -> é texto
else
    % Aplica Otsu no bloco
    y = im2bw(x, graythresh(x));
end
```



The local algoritmo 2

What is MATLAB?

MATLAB® is a high-performance language for technical computing. It integrates computation, visualization, and programming in an easy-to-use environment where problems and solutions are expressed in familiar mathematical notation. Typical uses include Math and computation Algorithm development Data acquisition Modeling simulation, and prototyping Data analysis exploration and visualization Scientific and engineering graphics Application development, including graphical user interface building

MATLAB is an interactive system whose basic data element is an array that does not require dimensioning. This allows you to solve many technical computing problems, especially those with matrix and vector formulations, in a fraction of the time it would take to write a program in a scalar noninteractive language such as C or Fortran.

The name MATLAB stands for matrix laboratory. MATLAB was originally written to provide easy access to matrix software developed by the LINPACK and EISPACK projects. Today, MATLAB engines incorporate the LAPACK and BLAS libraries, embedding the state of the art in software for matrix computation.

MATLAB has evolved over a period of years with input from many users. In university environments, it is the standard instructional tool for introductory and advanced courses in mathematics, engineering, and science. In industry, MATLAB is the tool of choice for high-productivity research, development, and analysis.

MATLAB features a family of add-on application-specific solutions called toolboxes. Very important to most users of MATLAB, toolboxes allow you to learn and apply specialized technology. Toolboxes are comprehensive collections of MATLAB functions (M-files) that extend the MATLAB environment to solve particular classes of problems. Areas in which toolboxes are available include signal processing, control systems, neural networks, fuzzy logic, wavelets, simulation, and many others.

The MATLAB System

The MATLAB system consists of five main parts:

Development Environment. This is the set of tools and facilities that help you use MATLAB functions and files. Many of these tools are graphical user interfaces. It includes the MATLAB desktop and Command Window, a command history editor and debugger, and browsers for viewing help, the workspace, files, and the search path.

The MATLAB Mathematical Function Library. This is a vast collection of computational algorithms ranging from elementary functions like sum, sine, cosine, and complex arithmetic, to more sophisticated functions like matrix inverse, matrix eigenvalues, Bessel functions, and fast Fourier transforms.

The MATLAB Language. This is a high-level matrix/array language with control flow statements, functions, data structures, input/output, and object-oriented programming features. It allows both "programming in the small" to rapidly create quick-and-dirty throw-away programs and "programming in the large" to create large and complex application programs.

Graphics. MATLAB has extensive facilities for displaying vectors and matrices as graphs, as well as annotating and printing these graphs. It includes high-level functions for two-dimensional and three-dimensional data visualization, image processing, animation, and presentation graphics. It also includes low-level functions that allow you to fully customize the appearance of graphics as well as to build complete graphical user interfaces on your MATLAB applications.

The MATLAB External Interfaces (API). This is a library that allows you to write C and Fortran programs that interact with MATLAB. It includes facilities for calling routines from MATLAB (dynamic linking), calling MATLAB as a computational engine, and for reading and writing MAT-files.

What is MATLAB?

MATLAB® is a high-performance language for technical computing. It integrates computation, visualization, and programming in an easy-to-use environment where problems and solutions are expressed in familiar mathematical notation. Typical uses include Math and computation Algorithm development Data acquisition Modeling simulation, and prototyping Data analysis exploration and visualization Scientific and engineering graphics Application development, including graphical user interface building

MATLAB is an interactive system whose basic data element is an array that does not require dimensioning. This allows you to solve many technical computing problems, especially those with matrix and vector formulations, in a fraction of the time it would take to write a program in a scalar noninteractive language such as C or Fortran.

The name MATLAB stands for matrix laboratory. MATLAB was originally written to provide easy access to matrix software developed by the LINPACK and EISPACK projects. Today, MATLAB engines incorporate the LAPACK and BLAS libraries, embedding the state of the art in software for matrix computation.

MATLAB has evolved over a period of years with input from many users. In university environments, it is the standard instructional tool for introductory and advanced courses in mathematics, engineering, and science. In industry, MATLAB is the tool of choice for high-productivity research, development, and analysis.

MATLAB features a family of add-on application-specific solutions called toolboxes. Very important to most users of MATLAB, toolboxes allow you to learn and apply specialized technology. Toolboxes are comprehensive collections of MATLAB functions (M-files) that extend the MATLAB environment to solve particular classes of problems. Areas in which toolboxes are available include signal processing, control systems, neural networks, fuzzy logic, wavelets, simulation, and many others.

The MATLAB system consists of five main parts:

Development Environment. This is the set of tools and facilities that help you use MATLAB functions and files. Many of these tools are graphical user interfaces. It includes the MATLAB desktop and Command Window, a command history editor and debugger, and browsers for viewing help, the workspace, files, and the search path.

The MATLAB Mathematical Function Library. This is a vast collection of computational algorithms ranging from elementary functions like sum, sine, cosine, and complex arithmetic, to more sophisticated functions like matrix inverse, matrix eigenvalues, Bessel functions, and fast Fourier transforms.

The MATLAB Language. This is a high-level matrix/array language with control flow statements, functions, data structures, input/output, and object-oriented programming features. It allows both "programming in the small" to rapidly create quick-and-dirty throw-away programs and "programming in the large" to create large and complex application programs.

Graphics. MATLAB has extensive facilities for displaying vectors and matrices as graphs, as well as annotating and printing these graphs. It includes high-level functions for two-dimensional and three-dimensional data visualization, image processing, animation, and presentation graphics. It also includes low-level functions that allow you to fully customize the appearance of graphics as well as to build complete graphical user interfaces on your MATLAB applications.

The MATLAB External Interfaces (API). This is a library that allows you to write C and Fortran programs that interact with MATLAB. It includes facilities for calling routines from MATLAB (dynamic linking), calling MATLAB as a computational engine, and for reading and writing MAT-files.

What is MATLAB?

MATLAB® is a high-performance language for technical computing. It integrates computation, visualization, and programming in an easy-to-use environment where problems and solutions are expressed in familiar mathematical notation. Typical uses include Math and computation Algorithm development Data acquisition Modeling simulation, and prototyping Data analysis exploration and visualization Scientific and engineering graphics Application development, including graphical user interface building

MATLAB is an interactive system whose basic data element is an array that does not require dimensioning. This allows you to solve many technical computing problems, especially those with matrix and vector formulations, in a fraction of the time it would take to write a program in a scalar noninteractive language such as C or Fortran.

The name MATLAB stands for matrix laboratory. MATLAB was originally written to provide easy access to matrix software developed by the LINPACK and EISPACK projects. Today, MATLAB engines incorporate the LAPACK and BLAS libraries, embedding the state of the art in software for matrix computation.

MATLAB has evolved over a period of years with input from many users. In university environments, it is the standard instructional tool for introductory and advanced courses in mathematics, engineering, and science. In industry, MATLAB is the tool of choice for high-productivity research, development, and analysis.

MATLAB features a family of add-on application-specific solutions called toolboxes. Very important to most users of MATLAB, toolboxes allow you to learn and apply specialized technology. Toolboxes are comprehensive collections of MATLAB functions (M-files) that extend the MATLAB environment to solve particular classes of problems. Areas in which toolboxes are available include signal processing, control systems, neural networks, fuzzy logic, wavelets, simulation, and many others.

The MATLAB System

The MATLAB system consists of five main parts:

Development Environment. This is the set of tools and facilities that help you use MATLAB functions and files. Many of these tools are graphical user interfaces. It includes the MATLAB desktop and Command Window, a command history editor and debugger, and browsers for viewing help, the workspace, files, and the search path.

The MATLAB Mathematical Function Library. This is a vast collection of computational algorithms ranging from elementary functions like sum, sine, cosine, and complex arithmetic, to more sophisticated functions like matrix inverse, matrix eigenvalues, Bessel functions, and fast Fourier transforms.

The MATLAB Language. This is a high-level matrix/array language with control flow statements, functions, data structures, input/output, and object-oriented programming features. It allows both "programming in the small" to rapidly create quick-and-dirty throw-away programs and "programming in the large" to create large and complex application programs.

Graphics. MATLAB has extensive facilities for displaying vectors and matrices as graphs, as well as annotating and printing these graphs. It includes high-level functions for two-dimensional and three-dimensional data visualization, image processing, animation, and presentation graphics. It also includes low-level functions that allow you to fully customize the appearance of graphics as well as to build complete graphical user interfaces on your MATLAB applications.

The MATLAB External Interfaces (API). This is a library that allows you to write C and Fortran programs that interact with MATLAB. It includes facilities for calling routines from MATLAB (dynamic linking), calling MATLAB as a computational engine, and for reading and writing MAT-files.

Pipeline complete para segmentação:

Imagem → Top-hat → Limiarização → Morfologia → Labeling

↑
morphology
gray-scale

2
2

5
eroso
dilated
opening
closing
opening-closing

↑